CISC 5800: Machine Learning
Lecture Notes

Brent Young*

1 Lecture 1: Course Introduction

January 22, 2024

1.1 Syllabus

1.1.1 Course Topics

1) Introduction to Machine Learning
2) Linear and Logistic Regression
3) Introduction to Neural Networks
4) Backpropagation in Neural Networks
5) Support Vector Machines (SVMs)
6) Clustering Techniques
7) Density Estimation
8) Noise Contrastive Estimation
9) Midterm Exam
10) Dimensionality Reduction

*Portions of these notes freely adapt material from references [1] and [2].

11) Expectation Maximization

12) Gaussian Mixture Models

13) Probabilistic Learning Models

14) Attention Networks and Large Language Models (LLMs)

1.1.2 Grading

Item	Percentage
Midterm	25%
Project	20%
Assignments	35%
Lecture Notes	10%
Participation	5%
Attendance	5%

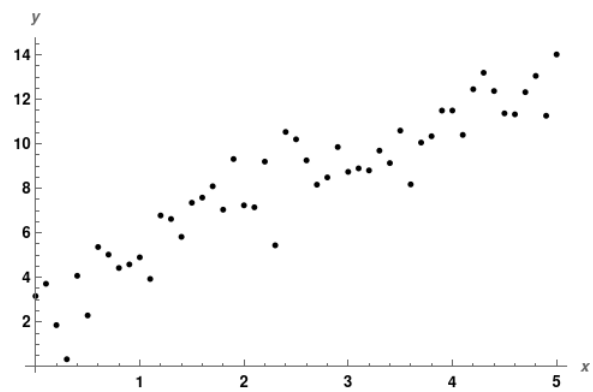
1.2 Introduction to Machine Learning

1.2.1 What is Machine Learning?

A traditional definition is that machine learning (ML) is concerned with the *automated discovery of patterns in data*. While this is a concise answer, it seems a little too ambitious since it is unclear that computers can really discover anything in a completely automated way. A better definition is:

> Given a particular objective, an ML model tries to optimize the feature weights that provide correct prediction.

As a simple example, consider the problem of trying to predict the dependent variable y for a given value of the independent variable x. The data consist of known pairs $\{(x_j, y_j)\}_{j=1}^{N}$ and are plotted below.



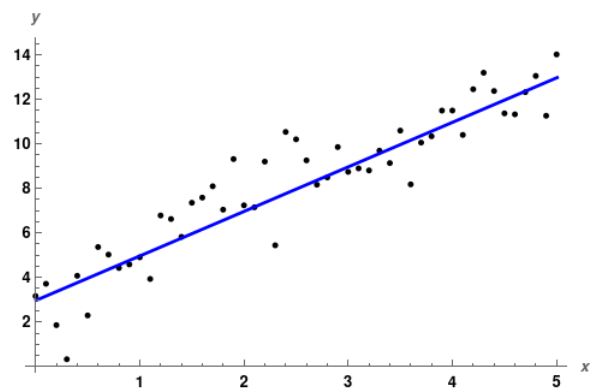
Given the data, it seems reasonable to assume there is a linear relationship between x and y; that is, $y_j = w_0 + w_1 x_j + \epsilon_j$ for some intercept w_0 and slope w_1 (ϵ_j represents noise or randomness in the data). But how can we determine the "best" line? After all, there is a line between any two of the data points. Or perhaps the best line does not actually go through any of the given points.

The idea is that we need some criterion which should characterize the best line. This criterion should hopefully give us an *objective function*, and optimizing this objective function should identify the best line (at least, the best line according to the chosen criterion).

Suppose we let $\hat{y}_j = w_0 + w_1 x_j$ be the value predicted by our (unknown) linear model for the given value x_j. One criterion we could use is that the best parameters should minimize the *sum of squared errors* (SSE):

$$\text{SSE} = \sum_{j=1}^{N} (\hat{y}_j - y_j)^2 = \sum_{j=1}^{N} (w_0 + w_1 x_j - y_j)^2 .$$

Minimizing the SSE over the variables w_0 and w_1 gives the predicted values $\hat{w}_0, \hat{w}_1$ which determine the best fit line for the model. This particular model is known as *linear regression*.



Of course, SSE is only one of an infinite number of objectives we could choose from (though it is a fairly common choice). For instance, we could have chosen to minimize the *sum of absolute deviations* (SAD):

$$\text{SAD} = \sum_{j=1}^{N} |\hat{y}_j - y_j| = \sum_{j=1}^{N} |w_0 + w_1 x_j - y_j| \,.$$

In general, minimizing SAD will lead to a different linear model than the usual linear regression.

1.2.2 Three Types of Machine Learning Models

1.2.3 Supervised Learning

In *supervised learning* we have a dataset where each entry has some number of features along with one or more labels. In large language models, the data may consist of prompt and response pairs. Our goal is to predict the label (or response) given the set of features (or given the prompt). Typically, the data are split into a training set (roughly 80% of the data) and a testing set (roughly 20%). The training set is used to determine the optimum feature weights, and the derived model is validated against the test data to give an idea of its accuracy.

Typically, supervised learning is easier to understand (though the algorithms may be complex). Common supervised learning models are Decision Trees, k–Nearest Neighbors, Naïve Bayes, and Support Vector Machines.

1.2.4 Unsupervised Learning

At the other extreme, there are *unsupervised learning* techniques which deal with completely unlabeled data. The goal is to determine patterns in the data without much of a preconceived notion of which points are similar. Unsupervised learning is generally more difficult than supervised, though there are a number of powerful algorithms that are relatively easy to understand. Some common unsupervised learning techniques are k–Means, Density–Based Clustering, Association Analysis, and Expectation Maximization.

1.2.5 Semi–Supervised Learning

It is possible that some data in the dataset are labeled while others are not. In this case, hybrid approaches, collectively known as *semi–supervised* learning, may be useful. One such technique is Topic Modeling where pieces of writing are sorted into groups based on their theme. The dataset may have plenty of examples of works pre–labeled as a particular genre. Other works may not be labeled, and we may not know all of the genres.

1.2.6 Methods of Deep Learning

There are a number of terms that come up regularly in the context of deep learning. *Reinforcement learning* is an ML model often used when the machine is interacting with an environment in order to achieve some goal. A system of penalties and rewards are used to train the system to perform the required task as efficiently as possible.

In large language models, an agent attempts to produce an accurate response based on a prompt given by the user. In an extreme case, the system must respond to a particular type of prompt without having any prior examples to work from. This is *zero–shot learning*. On the other hand, the system could be provided with sample prompt/response pairs for training. This is known as *few–shot learning*.

2 Lecture 2: Linear and Logistic Regression

January 29, 2024

2.1 Linear Regression

2.1.1 Linear Regression for One Independent Variable

Consider the problem of trying to find the best linear relationship for a given set of data $\{(x_i, y_i)\}_{i=1}^n$. That is, we want to reliably estimate the dependent value y_i given the corresponding value of the independent variable x_i. Our

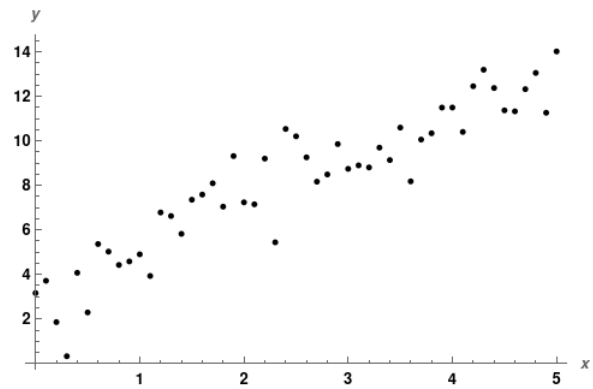


Figure 1: Sample Data for Linear Regression

goal is to find parameters ω_0 and ω_1 so that the predicted value

$$\hat{y}_i = \omega_0 + \omega_1 x_i$$

is a reasonable estimate of the true value y_i for all indices i.

As mentioned in the previous lecture, identifying the "best" linear relationship first requires us to determine what property (or properties) we expect this ideal linear function to possess. For linear regression, we want the line that minimizes the sum of squared errors:

$$\text{SSE} = \sum_{i=1}^{n} (\hat{y}_i - y_i)^2 = \sum_{i=1}^{n} (\omega_0 + \omega_1 x_i - y_i)^2.$$

Since the data set $\{(x_i, y_i)\}_{i=1}^n$ is given, the SSE is a function of the parameters ω_0 and ω_1. In order to identify a potential minimum, we must solve the

equations

$$\frac{\partial}{\partial \omega_0}(\text{SSE}) = 2\sum_{i=1}^{n}(\omega_0 + \omega_1 x_i - y_i) = 0 \tag{1}$$

$$\frac{\partial}{\partial \omega_1}(\text{SSE}) = 2\sum_{i=1}^{n}(\omega_0 + \omega_1 x_i - y_i)x_i = 0 \tag{2}$$

for the two parameters. We begin by solving (1) for ω_0.

$$\begin{aligned}
0 &= \sum_{i=1}^{n}(\omega_0 + \omega_1 x_i - y_i) \\
0 &= \omega_0 \sum_{i=1}^{n} 1 + \omega_1 \sum_{i=1}^{n} x_i - \sum_{i=1}^{n} y_i \\
n\omega_0 &= \sum_{i=1}^{n} y_i - \sum_{i=1}^{n} x_i \\
\omega_0 &= \bar{y} - \omega_1 \bar{x} \qquad (3)
\end{aligned}$$

In the final equation above, we substituted

$$\begin{aligned}
\bar{y} &= \frac{1}{n}\sum_{i=1}^{n} y_i \\
\bar{x} &= \frac{1}{n}\sum_{i=1}^{n} x_i
\end{aligned}$$

for the average values of the data.

Working with (2) (and substituting the result in (3)), we find

$$\begin{aligned}
\sum_{i=1}^{n}(\omega_0 x_i + \omega_1 x_i^2 - y_i x_i) &= 0 \\
(\bar{y} - \omega_1 \bar{x})\sum_{i=1}^{n} x_i + \omega_1 \sum_{i=1}^{n} x_i^2 - \sum_{i=1}^{n} x_i y_i &= 0 \\
\left(\sum_{i=1}^{n} x_i^2 - \bar{x}\sum_{i=1}^{n} x_i\right)\omega_1 &= \sum_{i=1}^{n} x_i y_i - \bar{y}\sum_{i=1}^{n} x_i \\
\omega_1 &= \frac{\text{Cov}(x, y)}{\text{Var}(x)} \qquad (4)
\end{aligned}$$

where we have used

$$\mathrm{Cov}(x,y) = \frac{1}{n}\sum_{i=1}^{n} x_i y_i - \bar{x}\bar{y},$$
$$\mathrm{Var}(x) = \frac{1}{n}\sum_{i=1}^{n} x_i^2 - \bar{x}^2.$$

To verify that these values of ω_0 and ω_1 minimize the sum of squared errors, we should technically run the second derivative test which involves checking the eigenvalues of the Hessian matrix of second partial derivatives. However, the sum of squared errors clearly has no maximum (as choosing a line $y = b + mx$ arbitrarily far from the data will give arbitrarily large error). Given the quadratic dependence on ω_0 and ω_1, we know that the critical values we have identified must give the minimum sum of squared errors.

2.1.2 Linear Regression in Higher Dimensions

If our data set is of the form $\{\mathbf{x}_i, y_i\}_{i=1}^n$ where

$$\mathbf{x}_i = \begin{bmatrix} x_{1,i} \\ x_{2,i} \\ \vdots \\ x_{m,i} \end{bmatrix}$$

is an m–dimensional feature vector, then we wish to find the best parameters $\omega_0, \omega_1, \cdots, \omega_m$ so that the value

$$\hat{y}_i = \omega_0 + \sum_{j=1}^{m} \omega_j x_{j,i}$$

is a reasonable estimate of the true value y_i. If we extend the feature vector to

$$\widetilde{\mathbf{x}}_i = \begin{bmatrix} 1 \\ x_{1,i} \\ x_{2,i} \\ \vdots \\ x_{m,i} \end{bmatrix},$$

we can write

$$\hat{y}_i = \begin{bmatrix} 1 & x_{1,i} & x_{2,i} & \cdots & x_{m,i} \end{bmatrix} \begin{bmatrix} \omega_0 \\ \omega_1 \\ \omega_2 \\ \vdots \\ \omega_m \end{bmatrix} = \widetilde{\mathbf{x}}_i^T \boldsymbol{\omega}.$$

We can collect the true values into a vector

$$\boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix},$$

and do the same for the estimates

$$\hat{\boldsymbol{y}} = \begin{bmatrix} \widetilde{\mathbf{x}}_i^T \boldsymbol{\omega} \\ \widetilde{\mathbf{x}}_2^T \boldsymbol{\omega} \\ \vdots \\ \widetilde{\mathbf{x}}_n^T \boldsymbol{\omega} \end{bmatrix}.$$

If we define the $(m+1) \times n$ matrix

$$\widetilde{\boldsymbol{X}} = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ x_{1,1} & x_{1,2} & \cdots & x_{1,n} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m,1} & x_{m,2} & \cdots & x_{m,n} \end{bmatrix},$$

then we can write

$$\hat{\boldsymbol{y}} = \widetilde{\boldsymbol{X}}^T \boldsymbol{\omega}.$$

Once again, we minimize the sum of squared errors.

$$\begin{aligned}
\text{SSE} &= \sum_{i=1}^{n} (\hat{y}_i - y_i)^2 = |\hat{\boldsymbol{y}} - \boldsymbol{y}|^2 \\
&= \left(\widetilde{\boldsymbol{X}}^T \boldsymbol{\omega} - \boldsymbol{y}\right)^T \left(\widetilde{\boldsymbol{X}}^T \boldsymbol{\omega} - \boldsymbol{y}\right) \\
&= \left(\boldsymbol{\omega}^T \widetilde{\boldsymbol{X}} - \boldsymbol{y}^T\right) \left(\widetilde{\boldsymbol{X}}^T \boldsymbol{\omega} - \boldsymbol{y}\right) \\
&= \boldsymbol{\omega}^T \widetilde{\boldsymbol{X}} \widetilde{\boldsymbol{X}}^T \boldsymbol{\omega} - \boldsymbol{y}^T \widetilde{\boldsymbol{X}}^T \boldsymbol{\omega} - \boldsymbol{\omega}^T \widetilde{\boldsymbol{X}} \boldsymbol{y} + \boldsymbol{y}^T \boldsymbol{y} \\
&= \boldsymbol{\omega}^T \widetilde{\boldsymbol{X}} \widetilde{\boldsymbol{X}}^T \boldsymbol{\omega} - 2\boldsymbol{\omega}^T \widetilde{\boldsymbol{X}} \boldsymbol{y} + \boldsymbol{y}^T \boldsymbol{y}
\end{aligned}$$

Taking the gradient with respect to the variables $\boldsymbol{\omega}$ gives

$$\boldsymbol{\nabla}_{\boldsymbol{\omega}} \text{SSE} = 2\widetilde{\boldsymbol{X}} \widetilde{\boldsymbol{X}}^T \boldsymbol{\omega} - 2\widetilde{\boldsymbol{X}} \boldsymbol{y}.$$

If we assume that the square matrix $\widetilde{\boldsymbol{X}} \widetilde{\boldsymbol{X}}^T$ is invertible, then setting the gradient equal to zero and solving for the parameters $\boldsymbol{\omega}$ gives

$$\boldsymbol{\omega} = \left(\widetilde{\boldsymbol{X}} \widetilde{\boldsymbol{X}}^T\right)^{-1} \widetilde{\boldsymbol{X}} \boldsymbol{y}.$$

If $\widetilde{\boldsymbol{X}} \widetilde{\boldsymbol{X}}^T$ is not invertible (for instance, if we have different y-values corresponding to the same feature vector), then the *Moore–Penrose pseudoinverse* can be used to compute the optimal weights:

$$\boldsymbol{\omega} = \left(\widetilde{X}^T\right)^{+} \boldsymbol{y}.$$

The pseudoinverse A^+ for any matrix A (not just square matrices) can be computed using the singular value decomposition (SVD) of matrix A.

2.2 Logistic Regression

If we have data where the dependent variable y_i is a binary label, then linear regression is not the most appropriate model. For such data, we could run a classification algorithm (such as Decision Trees, k–Nearest Neighbors, etc.). However, we could also try to attack the problem by regression. The basic idea is that we predict a value p in the range $0 < p < 1$ which we can interpret as the probability that an observation will be in the positive class. If the regression gives $p > 0.5$, we assume the instance is from the positive class.

For simplicity, we assume the data is of the form $\{(x_i, y_i)\}_{i=1}^n$ where the binary labels y_i are in the set $\{0, 1\}$. Our goal is to predict a probability p_i given x_i; $p_i < 0.5$ will predict a label of 0 while $p_i > 0$ will predict a label of 1. For any probability in the range $0 < p < 1$, the log–odds ratio

$$\text{logOdd}(p) = \ln\left(\frac{p}{1-p}\right)$$

will take on values in the interval $(-\infty, \infty)$. As such, we can imagine that these values might lie on a straight line.

$$\ln\left(\frac{p}{1-p}\right) = \omega_0 + \omega_1 x$$

Solving this for p gives a function of the form

$$p = \frac{1}{1 + e^{-(\omega_0 + \omega_1 x)}}$$

which has a sigmoid shape: it limits to 0 as $x \to -\infty$, to 1 as $x \to \infty$, and is strictly increasing over the real number line. Note that the form of this curve is the same as the logistic growth curve from population modeling. As such, fitting such a curve to data is known as *logistic regression*.

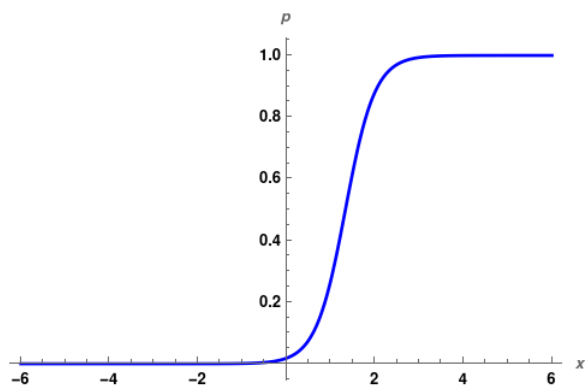


Figure 2: Typical Sigmoid Curve

Now that we have the form of the curve, the problem turns to identifying the best such curve to fit the data. In general, we might think to use linear regression on the data set

$$\left\{\left(\ln\left(\frac{y_i}{1-y_i}\right), x_i\right)\right\}_{i=1}^{n}.$$

If the y_i were actually continuous values strictly in the interval $(0, 1)$, this might be a valid idea. However, our y_i are the binary labels 0 and 1, and the log–odds ratio is not well defined for these values. So, we need to identify some other criterion for the optimal fit.

The key idea is to choose a *cost function* to take the place of the sum of squared errors, and we then try to identify the parameters ω_0 and ω_1 that minimize the total cost. Let

$$\hat{y}_i = \frac{1}{1 + e^{-(\omega_0 + \omega_1 x_i)}}$$

be the value predicted by our logistic regression model for input x_i. The cost function typically used is

$$c(\hat{y}_i, y_i) = -\ln(\hat{y}_i)(y_i) - \ln(1 - \hat{y}_i)(1 - y_i).$$

To understand why this is a good cost function, first imagine that the true label is $y_i = 0$.

$$c(\hat{y}_i, 0) = -\ln(1 - \hat{y}_i)$$

If $\hat{y}_i$ is fairly close to the true value 0, then the overall cost will be reasonably close to 0. On the other hand,

$$\lim_{\hat{y}_i \to 1^-} -\ln(1-\hat{y}_i) = \infty,$$

and the cost for predictions closer to 1 becomes very large. The reverse is true if the true label is $y_i = 1$.

$$c(\hat{y}_i, 1) = -\ln(\hat{y}_i)$$

Now, if $\hat{y}_i$ is close to zero, the cost will be very large (and $\hat{y}_i \approx 1$ gives a cost close to 0).

The task in logistic regression is to find optimum parameters ω_0 and ω_1 so that the predictions

$$\hat{y}_i = \frac{1}{1+e^{-(\omega_0+\omega_1 x_i)}}$$

minimize the total cost function

$$C(\hat{\boldsymbol{y}}, \boldsymbol{y}) = \frac{1}{n}\sum_{i=1}^{n} c(\hat{y}_i, y_i).$$

Unfortunately, the equation

$$\boldsymbol{\nabla}_{\boldsymbol{\omega}} C(\hat{\boldsymbol{y}}, \boldsymbol{y}) = \mathbf{0}$$

cannot be explicitly solved for the parameters. Instead, we estimate the optimal parameters using the method of *gradient descent*. We record the partial derivatives below for reference.

$$\begin{aligned}
\frac{\partial}{\partial \omega_0}\left[C(\hat{\boldsymbol{y}}, \boldsymbol{y})\right] &= \frac{1}{n}\sum_{i=1}^{n} \frac{\partial}{\partial \omega_0}\left[c(\hat{y}_i, y_i)\right] \\
&= \frac{1}{n}\sum_{i=1}^{n} \frac{\partial}{\partial \hat{y}_i}\left[c(\hat{y}_i, y_i)\right] \frac{\partial \hat{y}_i}{\partial \omega_0} \\
&= \frac{1}{n}\sum_{i=1}^{n} \left(\frac{1-y_i}{1-\hat{y}_i} - \frac{y_i}{\hat{y}_i}\right)\left(\frac{e^{-(\omega_0+\omega_1 x_i)}}{\left(1+e^{-(\omega_0+\omega_1 x_i)}\right)^2}\right) \\
&= \frac{1}{n}\sum_{i=1}^{n} \left(\frac{1-y_i}{1-\hat{y}_i} - \frac{y_i}{\hat{y}_i}\right)\hat{y}_i(1-\hat{y}_i) \\
&= \frac{1}{n}\sum_{i=1}^{n} \left(\hat{y}_i - y_i\right)
\end{aligned}$$

$$
\begin{aligned}
\frac{\partial}{\partial \omega_1}\left[C(\hat{\boldsymbol{y}}, \boldsymbol{y})\right] &= \frac{1}{n}\sum_{i=1}^{n} \frac{\partial}{\partial \hat{y}_i}\left[c(\hat{y}_i, y_i)\right] \frac{\partial \hat{y}_i}{\partial \omega_1} \\
&= \frac{1}{n}\sum_{i=1}^{n} \left(\frac{1-y_i}{1-\hat{y}_i} - \frac{y_i}{\hat{y}_i}\right)\left(\frac{e^{-(\omega_0+\omega_1 x_i)} x_i}{\left(1+e^{-(\omega_0+\omega_1 x_i)}\right)^2}\right) \\
&= \frac{1}{n}\sum_{i=1}^{n} \left(\frac{1-y_i}{1-\hat{y}_i} - \frac{y_i}{\hat{y}_i}\right) \hat{y}_i(1-\hat{y}_i) x_i \\
&= \frac{1}{n}\sum_{i=1}^{n} \left(\hat{y}_i - y_i\right) x_i
\end{aligned}
$$

A typical result of logarithmic regression is shown below.

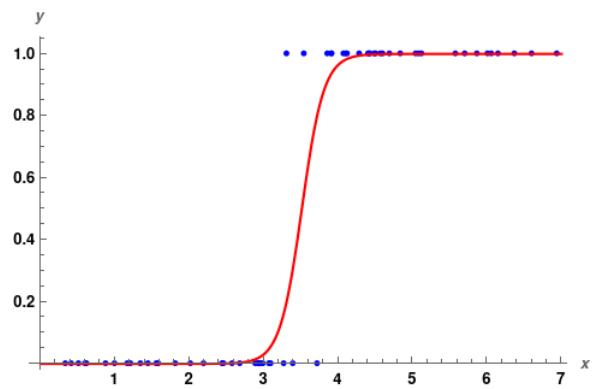


Figure 3: Typical Logarithmic Regression

3 Lecture 3: Logistic Regression and Gradient Descent

February 5, 2024

3.1 More on Logistic Regression

As we saw in the last section, the best coefficients for Logistic Regression can be obtained by solving

$$\boldsymbol{\nabla}_{\boldsymbol{\omega}} C(\hat{\boldsymbol{y}}, \boldsymbol{y}) = \mathbf{0}$$

for the parameters ω_0 and ω_1 that minimize the overall cost function. Here, $\hat{\boldsymbol{y}} = (\hat{y}_1, \hat{y}_2, \cdots, \hat{y}_n)$ is the vector of predictions

$$\hat{y}_i = \frac{1}{1 + e^{-(\omega_0 + \omega_1 x_i)}},$$

and $\boldsymbol{y} = (y_1, y_2, \cdots, y_n)$ is the vector of true labels (0 or 1) corresponding to the value of the independent variable x_i.

Unlike the case of linear regression, solving these equations for the ideal parameters is practically impossible. As such, we need to resort to an approximation scheme in order to find values of ω_0 and ω_1 that are at least reasonably close to the true minimizers. Before moving on to discuss the approximation scheme, we re-derive the relevant formulae for a vector of independent variables.

Let the given data be represented be pairs $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$. Here y_i is still a binary label (0 or 1), but the

$$\boldsymbol{x}_i = \begin{bmatrix} 1 \\ x_{1,i} \\ x_{2,i} \\ \vdots \\ x_{m,i} \end{bmatrix}$$

is an $m+1$-dimensional vector of the independent variables (with the extra 1 for convenience just like in Linear Regression). If we write the vector of

parameters

$$\boldsymbol{\theta} = \begin{bmatrix} \omega_0 \\ \omega_1 \\ \omega_2 \\ \vdots \\ \omega_m \end{bmatrix},$$

then the predicted values can be written

$$\hat{y}_i = \frac{1}{1 + e^{-(\omega_0 + \omega_1 x_{1,i} + \cdots + \omega_m x_{m,i})}} = \frac{1}{1 + e^{-\boldsymbol{\theta}^T \boldsymbol{x}_i}}$$

As before, the cost function for this model is

$$\begin{aligned} C(\hat{\boldsymbol{y}}, \boldsymbol{y}) &= \frac{1}{n} \sum_{i=1}^{n} c(\hat{y}_i, y_i) \\ &= \frac{1}{n} \sum_{i=1}^{n} \big(- y_i \ln(\hat{y}_i) - (1 - y_i) \ln(1 - \hat{y}_i) \big), \end{aligned}$$

and the dependence on the parameters in $\boldsymbol{\theta}$ is only through the predicted values $\hat{y}_i$.

The gradient of the cost function can be computed using the Chain Rule.

$$\boldsymbol{\nabla}_{\boldsymbol{\theta}} C(\hat{\boldsymbol{y}}, \boldsymbol{y}) = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{\nabla}_{\boldsymbol{\theta}} c(\hat{y}_i, y_i) = \frac{1}{n} \sum_{i=1}^{n} \frac{\partial}{\partial \hat{y}_i} c(\hat{y}_i, y_i) \boldsymbol{\nabla}_{\boldsymbol{\theta}} \hat{y}_i$$

As before,

$$\begin{aligned} \frac{\partial}{\partial \hat{y}_i} c(\hat{y}_i, y_i) &= -\frac{y_i}{\hat{y}_i} + \frac{1 - y_i}{1 - \hat{y}_i} \\ &= \frac{(1 - y_i)\hat{y}_i - y_i(1 - \hat{y}_i)}{\hat{y}_i(1 - \hat{y}_i)} \\ &= \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)}. \end{aligned}$$

For the remaining term in the Chain Rule, we note that if $\sigma(z) = \dfrac{1}{1 + e^{-z}}$,

then

$$\begin{aligned}
\sigma'(z) &= \frac{e^{-z}}{(1+e^{-z})^2} \\
&= \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} \\
&= \sigma(z) \cdot \left(\frac{1+e^{-z}}{1+e^{-z}} - \frac{1}{1+e^{-z}} \right) \\
&= \sigma(z)\,(1-\sigma(z))\,.
\end{aligned}$$

This fact allows us to compute the final derivative very easily.

$$\begin{aligned}
\boldsymbol{\nabla}_{\boldsymbol{\theta}} \hat{y}_i &= \boldsymbol{\nabla}_{\boldsymbol{\theta}} \sigma(\boldsymbol{\theta}^T \boldsymbol{x}_i) \\
&= \sigma'(\boldsymbol{\theta}^T \boldsymbol{x}_i) \boldsymbol{\nabla}_{\boldsymbol{\theta}} \left(\boldsymbol{\theta}^T \boldsymbol{x}_i \right) \\
&= \sigma(\boldsymbol{\theta}^T \boldsymbol{x}_i)(1-\sigma(\boldsymbol{\theta}^T \boldsymbol{x}_i))\boldsymbol{x}_i \\
&= \hat{y}_i(1-\hat{y}_i)\boldsymbol{x}_i
\end{aligned}$$

Finally, we see that

$$\boldsymbol{\nabla}_{\boldsymbol{\theta}} C(\hat{\boldsymbol{y}}, \boldsymbol{y}) = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)\boldsymbol{x}_i.$$

For the individual partial derivatives, we have the following.

$$\begin{aligned}
\frac{\partial}{\partial \omega_0} C(\hat{\boldsymbol{y}}, \boldsymbol{y}) &= \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i) \\
\frac{\partial}{\partial \omega_j} C(\hat{\boldsymbol{y}}, \boldsymbol{y}) &= \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i) x_{j,i} \quad (1 \le j \le m)
\end{aligned}$$

3.2 Gradient Descent

In order to understand the method of Gradient Descent, we first need to discuss the information conveyed by the gradient of a function. For simplicity, we consider a function of two variables $f = f(x, y)$. For our purposes, the best way to visualize the function f is a *contour plot*. This is exactly like a topographic map of elevation; the curves $f(x, y) = c$ are drawn for various values of c, and the family of these curves is displayed in the xy–plane marked by the value of c.

In the contour plot below, the contours have been color–coded to make the graph of the function easier to understand. The blue regions show where the values of f are positive, red regions are where f is negative, and the lightly colored regions are where the values of f are close to 0. If we think of the contour plot as a map of elevation, then the landscape would look like a hill in the center of the graph surrounded by four valleys.

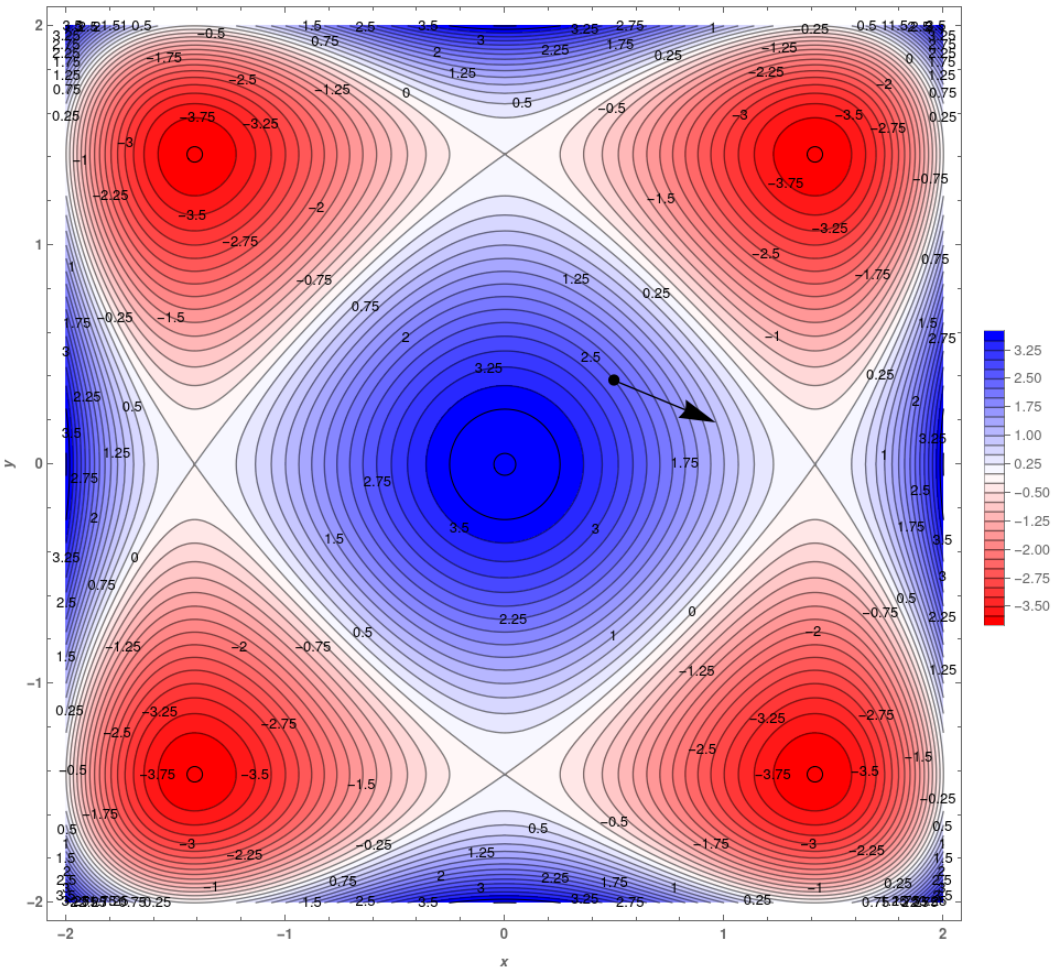


Figure 4: Sample Contour Plot

A natural question to ask is what happens to the value of f if we start at

some point (x_0, y_0) and begin moving in the direction given by a unit vector

$$\hat{\boldsymbol{u}} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}.$$

In other words, we want the value of $D_{\hat{\boldsymbol{u}}} f(x_0, y_0)$, the *directional derivative* of f at the point (x_0, y_0) in the direction $\hat{\boldsymbol{u}}$. In the contour plot above, we would expect the directional derivative at the given point in the direction of the displayed arrow to be negative since we are going downhill if we move in that direction.

The directional derivative is completely determined by the gradient of f at the point (x_0, y_0).

$$D_{\hat{\boldsymbol{u}}} f(x_0, y_0) = \boldsymbol{\nabla} f(x_0, y_0) \cdot \hat{\boldsymbol{u}} = \begin{bmatrix} \frac{\partial f}{\partial x}(x_0, y_0) \\ \frac{\partial f}{\partial x}(x_0, y_0) \end{bmatrix} \cdot \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = u_1 \frac{\partial f}{\partial x}(x_0, y_0) + u_2 \frac{\partial f}{\partial y}(x_0, y_0)$$

To understand the significance of the gradient, we use the fact that dot products are related to the angle φ between two vectors (with $0 \leq \varphi \leq \pi$).

$$D_{\hat{\boldsymbol{u}}} f(x_0, y_0) = \boldsymbol{\nabla} f(x_0, y_0) \cdot \hat{\boldsymbol{u}} = |\boldsymbol{\nabla} f(x_0, y_0)|\, |\hat{\boldsymbol{u}}| \cos(\varphi) = |\boldsymbol{\nabla} f(x_0, y_0)| \cos(\varphi)$$

Note that the last equality above is the only place where $\hat{\boldsymbol{u}}$ being a unit vector is significant to the discussion.

To maximize the directional derivative at a given point, we should choose the unit vector *in the same direction as the gradient* (so that $\varphi = 0$ and $\cos(\varphi) = 1$). This tells us that $\boldsymbol{\nabla} f(x_0, y_0)$ points in the direction where f *will increase at the fastest possible rate* at the point (x_0, y_0). Moreover, the magnitude of the gradient gives us the maximum possible rate of change. As the plot below clearly shows, the gradient vector (shown in green) is pointing directly uphill.

We also see that $-\boldsymbol{\nabla} f(x_0, y_0)$ points in the direction where f *will decrease at the fastest possible rate* at the point (x_0, y_0). This gives us another way to see that the directional derivative of f in the direction of the black arrow is negative; the black arrow points more in the direction opposite to $\boldsymbol{\nabla} f(x_0, y_0)$, and as a result the values of f should decrease in that direction.

If the solutions to

$$\boldsymbol{\nabla} f(x, y) = \mathbf{0}$$

are too difficult to solve directly, the geometric interpretation of the gradient suggests a convenient way to search for local maxima and minima. The

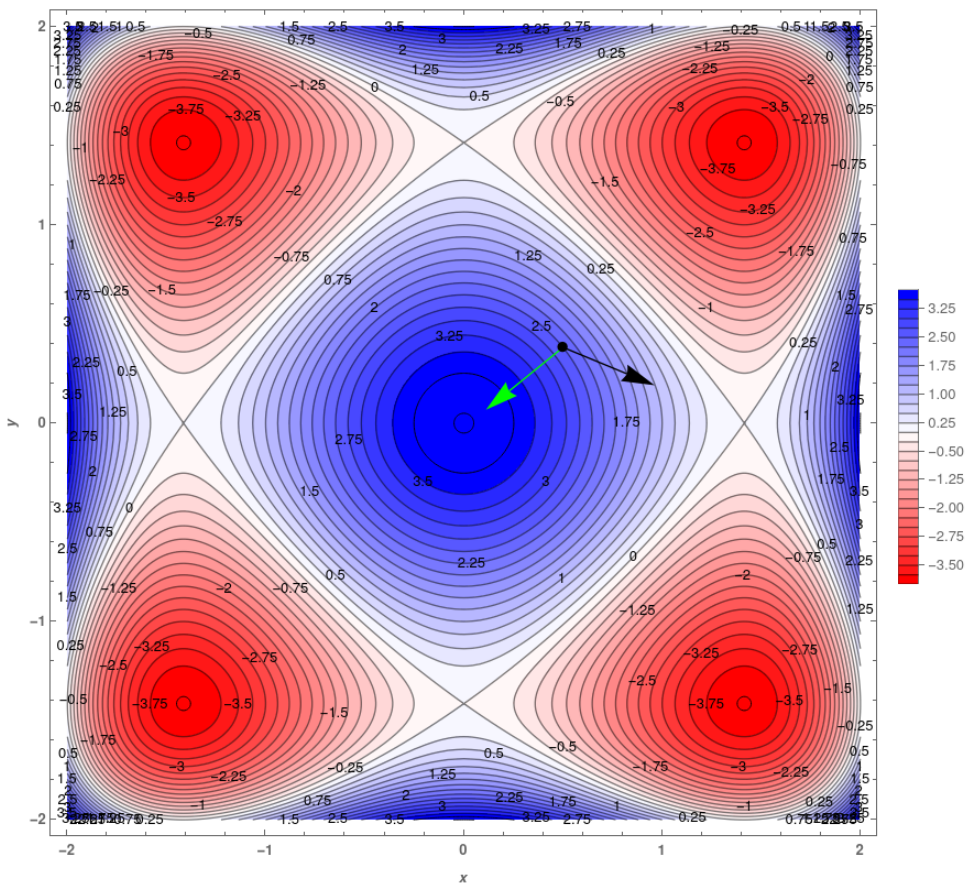


Figure 5: Sample Contour Plot (normalized gradient vector shown in green)

idea is to select a random initial point (x_0, y_0) and compute the gradient $\boldsymbol{\nabla} f(x_0, y_0)$. We also select a parameter $\eta > 0$ usually taken to be small. If we want to move toward a local maximum, *we move in the direction of the gradient*

$$\begin{bmatrix} x_n \\ y_n \end{bmatrix} = \begin{bmatrix} x_{n-1} \\ y_{n-1} \end{bmatrix} + \eta \boldsymbol{\nabla} f(x_{n-1}, y_{n-1})$$

repeating this process until the change in the coordinates drops below some established threshold. This process is known as *gradient ascent*.

More commonly, f represents some cost function that we want to mini-

mize. In that case, we need to *move in the opposite direction of the gradient.*

$$\begin{bmatrix} x_n \\ y_n \end{bmatrix} = \begin{bmatrix} x_{n-1} \\ y_{n-1} \end{bmatrix} - \eta \boldsymbol{\nabla} f(x_{n-1}, y_{n-1})$$

Unsurprisingly, this technique is known as *gradient descent*, and the process is repeated until the coordinates stabilize (again within some threshold).

As our plot above shows, f could have multiple local maxima and minima. If we want to identify several different critical points, we can randomly choose multiple starting points (x_0, y_0), run either gradient ascent or descent (depending on the application) for each starting point, and examine the locations identified by each run. This should produce several clusters of points around the various extrema.

3.3 Algorithm for Logistic Regression

Coming back to the problem of logistic regression, we assume we are given data $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ with y_i a binary label (either 0 or 1) and

$$\boldsymbol{x}_i = \begin{bmatrix} 1 \\ x_{1,i} \\ x_{2,i} \\ \vdots \\ x_{m,i} \end{bmatrix}.$$

Our vector of parameters is

$$\boldsymbol{\theta} = \begin{bmatrix} \omega_0 \\ \omega_1 \\ \omega_2 \\ \vdots \\ \omega_m \end{bmatrix},$$

and the predicted values (for given values of the parameters) can be written

$$\hat{y}_i = \frac{1}{1 + e^{-(\omega_0 + \omega_1 x_{1,i} + \cdots + \omega_m x_{m,i})}} = \frac{1}{1 + e^{-\boldsymbol{\theta}^T \boldsymbol{x}_i}}.$$

It can be shown that the cost function

$$C(\hat{\boldsymbol{y}}, \boldsymbol{y}) = \frac{1}{n} \sum_{i=1}^{n} \big(- y_i \ln(\hat{y}_i) - (1 - y_i) \ln(1 - \hat{y}_i) \big)$$

is convex. This implies that if it has a minimum, that minimum is a global minimum (and so gradient descent should work well).

In the following algorithm, we set a learning rate $\eta > 0$ which controls how far we follow the negative gradient at each step and a threshold $\epsilon > 0$ which tells the algorithm when to stop.

```
eta = 0.001
espilon = 0.000001

def sigmoid(x, theta):
    arg = 0

    for j in range(len(theta)):
        arg += x[j]*theta[j]

    return 1 / (1 + exp(-arg))

def logLoss(X, y, theta):
    n = len(y)
    ret = 0

    for i in range(n):
        ret += -y[i]*log(sigmoid(X[i], theta))
        ret += - (1-y[i])*log(1-sigmoid(X[i], theta))

    return ret / n

def gradLogLoss(X, y, theta):
    n = len(y)

    yhat = []
    for i in range(n):
        yhat.append(sigmoid(X[i], theta))

    ret = []
    for j in range(len(theta)):
        tmp = 0
        for i in range(n):
            tmp += (yhat[i] - y[i])*X[i][j]
        ret.append(tmp/n)

    return ret
```

```
def gradientDescent(X, y, eta, epsilon):
    # X = (supplied data)
    # y = (binary labels)

    # J will be the number of independent variables
    # (including the leading 1 for w0)
    J = len(X[0])

    # Initialize theta
    theta = []
    for j in range(J):
        theta.append(random.uniform(-10,10))

    # Gradient Descent
    cost = logLoss(X, y, theta)
    delta = 1

    while delta > epsilon:
        new = []
        grad = gradLogLoss(X, y, theta)
        for j in range(J):
            new.append(theta[j] - eta*grad[j])
        theta = new
        new_cost = logLoss(X, y, theta)
        delta = abs(new_cost - cost)
        cost = new_cost

    return theta
```

4 Lecture 4: Neural Nets

February 11, 2024

4.1 A Single Neuron

We can think of our logistic regression problem as a mathematical model of a neuron.

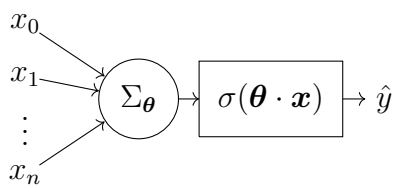
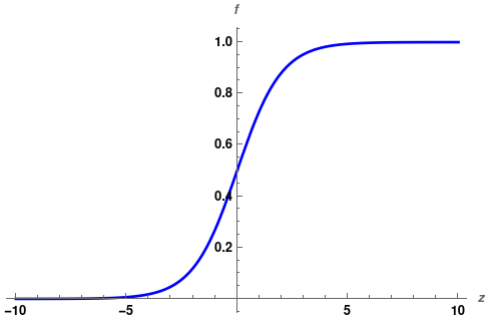


Figure 6: An Artificial Neuron

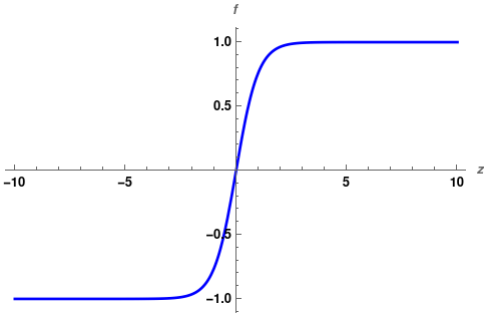
First, our artificial neuron combines the inputs $\boldsymbol{x} = [x_0, x_1, \cdots, x_n]^T$ using the weights $\boldsymbol{\theta} = [\omega_0, \omega_1, \cdots, \omega_n]^T$ (there could be rescaling at this step if necessary). The weighted inputs are then used as input to our logistic sigmoid function to produce the output $\hat{y}$. We think of the logistic sigmoid as an *activation function*. Real neurons behave in an analogous manner. They receive inputs (from other neurons), and the output from the neuron is a function of the strength of the inputs. Some common choices of activation functions and their graphs are shown in the figures below.

Activation Function	Formula
Logistic Sigmoid	$\sigma(z) = \dfrac{1}{1+e^{-z}}$
Hyperbolic Tangent	$\tanh(z) = \dfrac{e^z - e^{-z}}{e^z + e^{-z}}$
Rectified Linear Unit	$\mathrm{ReLU}(z) = \max\{0, x\}$
Exponential Linear Unit	$\mathrm{ELU}(z) = \begin{cases} \alpha(e^z - 1), & z < 0 \\ z, & z \geq 0 \end{cases}$
Scaled Exponential Unit	$\mathrm{SEU}(z) = \dfrac{1}{\lambda} \ln\left(1 + e^{\lambda z}\right)$
Swish Activation Function	$f(z) = z\sigma(\beta z)$

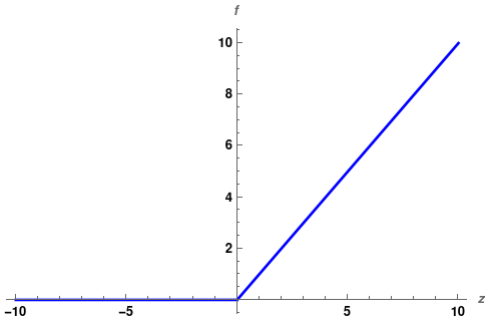
Figure 7: Common Activation Functions



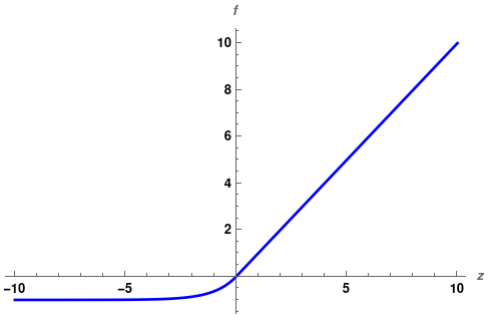
(a) Logistic Sigmoid



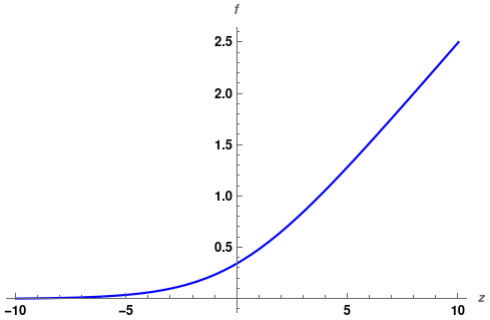
(b) Hyperbolic Tangent



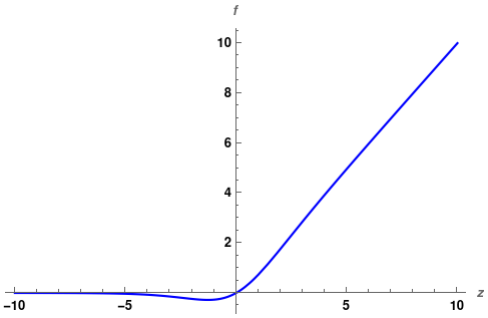
(c) Rectified Linear Unit



(d) Exponential Linear Unit



(e) Scaled Exponential Unit



(f) Swish Activation Function

Figure 8: Graphs of Common Activation Functions

4.2 Neural Nets

4.2.1 Single–Layer Perceptron

Suppose we have a classification problem with multiple labels. If there are m labels, we take inspiration from the idea of one–hot encoding and imagine we have m dependent variables y_j that are either 0 or 1 depending on whether the example has label j. If our independent variables are $\boldsymbol{x} = [x_1, x_2, \cdots, x_d]^T$, then the architecture of the perceptron is as shown below.

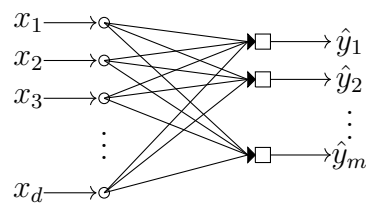


Figure 9: Single–Layer Perceptron Architecture

Each $\hat{y}_j$ is a sigmoid function of the inputs:

$$\hat{y}_j = \sigma(\boldsymbol{\theta}_j \cdot \boldsymbol{x}) = \sigma(\omega_{j,1}x_1 + \omega_{j,2}x_2 + \omega_{j,d}x_d)$$

where $\boldsymbol{\theta}_j = [\omega_{j,1}, \omega_{j,2}, \cdots, \omega_{j,d}]^T$. The notation is set up so that $\omega_{j,k}$ is the weight that output $\hat{y}_j$ assigns to input x_k. As with a single neuron, we use the log–loss function to identify the optimal weights. For a single example $\boldsymbol{x}_i$ with known labels $\boldsymbol{y}_i$, the loss is

$$L(\hat{\boldsymbol{y}}_i, \boldsymbol{y}_i) = \sum_{j=1}^{m} \left[-y_{i,j} \ln(\hat{y}_{i,j}) - (1 - y_{i,j}) \ln(1 - \hat{y}_{i,j})\right].$$

Summing these over all examples gives the total cost.

$$C(\hat{\boldsymbol{y}}, \boldsymbol{y}) = \frac{1}{n} \sum_{i=1}^{n} L(\hat{\boldsymbol{y}}_i, \boldsymbol{y}_i)$$

Since this is a positive linear combination of convex functions with strictly positive coefficients, the cost function above is itself convex. As such, we can expect there to be a unique minimum which we can approximate using gradient descent.

To update a particular $\omega_{j,k}$, we first note that this weight only appears in the calculation of $\hat{y}_{i,j}$.

$$\begin{aligned}
\frac{\partial C}{\partial \omega_{j,k}} &= \frac{1}{n}\sum_{i=1}^{n} \frac{\partial L}{\partial \hat{y}_{i,j}} \frac{\partial \hat{y}_{i,j}}{\partial \omega_{j,k}} \\
&= \frac{1}{n}\sum_{i=1}^{n} \left(\frac{\hat{y}_{i,j} - y_{i,j}}{\hat{y}_{i,j}(1-\hat{y}_{i,j})} \right) \hat{y}_{i,j}(1-\hat{y}_{i,j}) x_{i,k} \\
&= \frac{1}{n}\sum_{i=1}^{n} (\hat{y}_{i,j} - y_{i,j}) x_{i,k}
\end{aligned}$$

For each training example i (with $1 \leq i \leq n$), we update all the weights by

$$\omega_{j,k} \longleftarrow \omega_{j,k} + \frac{\eta}{n}(y_{i,j} - \hat{y}_{i,j}) x_{i,k}$$

where $1 \leq j \leq m$, $1 \leq k \leq d$, and $\eta > 0$ is the learning rate.

4.2.2 Multilayer Perceptron

To improve on the single–layer perceptron, we add a number of "hidden layers" between the inputs and the outputs. We can think of this as a series of single–layer perceptrons wired together so that the output of one layer becomes the input of the next.

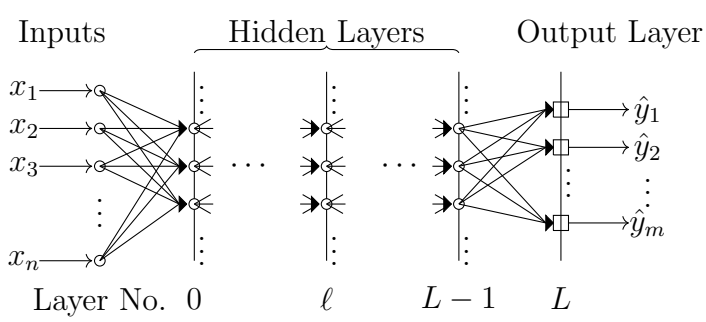


Figure 10: Multilayer Perceptron Architecture

We refer to the nodes in hidden layer ℓ as $h^{\ell}_{i,j}$ where $1 \leq j \leq n_\ell$ with n_ℓ the total number of nodes in the hidden layer. The index i refers to the value of the node produced from input $\boldsymbol{x}_i$. The total number of these layers will

be L. Our loss and total cost functions are the same as in the single–layer perceptron.

$$L(\hat{\boldsymbol{y}}_{\boldsymbol{i}}, \boldsymbol{y}_i) = \sum_{j=1}^{m} \left[-y_{i,j} \ln(\hat{y}_{i,j}) - (1 - y_{i,j}) \ln(1 - \hat{y}_{i,j})\right]$$

$$C(\hat{\boldsymbol{y}}, \boldsymbol{y}) = \frac{1}{n} \sum_{i=1}^{n} L(\hat{\boldsymbol{y}}_i, \boldsymbol{y}_i)$$

Note that neural networks can also be used for regression tasks (and so other cost functions such as mean squared error might be more appropriate). As before, we randomly choose weights for each layer, compute the predicted values, and update the weights based on the gradient of the cost function for our current list of weights.

The primary issue now is that there are many more weights in the model, and they appear in the loss function in complicated ways. For the outputs,

$$\hat{y}_{i,j} = \sigma\left(\boldsymbol{\theta}_j^L \cdot \boldsymbol{h}_i^{L-1}\right) = \sigma\left(\omega_{j,1}^L h_{i,1}^{L-1} + \omega_{j,2}^L h_{i,2}^{L-1} + \cdots + \omega_{j,n_L}^L h_{i,n_L}^{L-1}\right).$$

So, the partial derivatives of the weights for the final layer, $\omega_{j,k}^L$, are just as easy to compute as for the single–layer perceptron. The complication is that the value $h_{i,k}^L$ depends on the values from the previous layer with their own set of weights. For instance,

$$h_{i,k}^{L-1} = \sigma\left(\boldsymbol{\theta}_k^{L-1} \cdot \boldsymbol{h}_i^{L-2}\right) = \sigma\left(\omega_{k,1}^{L-1} h_{i,1}^{L-2} + \omega_{k,2}^{L-1} h_{i,2}^{L-2} + \cdots + \omega_{k,n_{L-1}}^{L-1} h_{i,n_{L-1}}^{L-2}\right).$$

This pattern generalizes for the internal layers.

$$h_{i,j}^{\ell} = \sigma\left(\boldsymbol{\theta}_j^{\ell} \cdot \boldsymbol{h}_i^{\ell-1}\right) = \sigma\left(\omega_{j,1}^{\ell} h_{i,1}^{\ell-1} + \omega_{j,2}^{\ell} h_{i,2}^{\ell-1} + \cdots + \omega_{j,n_{\ell-1}}^{\ell} h_{i,n_{\ell-1}}^{\ell-1}\right)$$

The final layer is computed directly from the inputs.

$$h_{i,j}^0 = \sigma\left(\boldsymbol{\theta}_j^0 \cdot \boldsymbol{x}_i\right) = \sigma\left(\omega_{j,1}^0 x_{i,1} + \omega_{j,2}^0 x_{i,2} + \cdots + \omega_{j,n}^0 x_{i,d}\right)$$

For any of the weights, gradient descent requires

$$\omega_{j,k}^{\ell} \longleftarrow \omega_{j,k}^{\ell} - \eta \frac{\partial C}{\partial \omega_{j,k}^{\ell}} = \omega_{j,k}^{\ell} - \frac{\eta}{n} \sum_{i=1}^{n} \frac{\partial L(\hat{\boldsymbol{y}}_i, \boldsymbol{y}_i)}{\partial \omega_{j,k}^{\ell}}$$

for a reasonable choice of learning rate $\eta > 0$. Computation of the partial derivatives of the loss function is no longer a trivial task. One way of efficiently organizing the work is *back propagation*.

4.3 Back Propagation for Sigmoid Activation and Log–Loss Cost Function

As mentioned above, we first randomly choose the initial weights for each layer. Once selected, we use the input vector $\boldsymbol{x}_i$ to make final predictions $\hat{\boldsymbol{y}}_i$ by running through all computations in the hidden layers. This process of going from inputs to outputs with specified weights is called *forward propagation*. Once we perform the forward propagation phase, we can compute the total cost and determine if updates to the weights are required. Since the loss function for each example $(\boldsymbol{x}_i, \boldsymbol{y}_i)$ is identical, we perform the same basic computation for each data point. Below, we suppress the index i for ease of reading. That is to say, we focus on updating the weights for a single generic example $(\boldsymbol{x}, \boldsymbol{y})$.

The process of successively computing updates to the weights is called back propagation (or *backprop* for short). As we saw above, the main difficulty is that all of the weights before the final layer are nested inside of compositions of our logistic sigmoid $\sigma(z)$. The final layer of weights is the simplest.

$$\begin{aligned}
\frac{1}{n}\frac{\partial L}{\partial \omega_{j,k}^L} &= \frac{1}{n}\frac{\partial}{\partial \omega_{j,k}^L}\sum_{p=1}^{m}\left[-y_p \ln(\hat{y}_p) - (1-y_p)\ln(1-\hat{y}_p)\right] \\
&= \frac{1}{n}\left[-\frac{y_j}{\hat{y}_j} + \frac{1-y_j}{1-\hat{y}_j}\right]\frac{\partial \hat{y}_j}{\partial \omega_{j,k}^L} \\
&= \frac{1}{n}\frac{\hat{y}_j - y_j}{\hat{y}_j(1-\hat{y}_j)}\hat{y}_j(1-\hat{y}_j)h_k^{L-1} \\
&= \frac{1}{n}(\hat{y}_j - y_j)h_k^{L-1}
\end{aligned}$$

This shows us how to update the weights of the outermost layer for a single training example *at the end of the entire backprop process.*

$$\omega_{j,k}^L \longleftarrow \omega_{j,k}^L - \frac{\eta}{n}\frac{\partial C}{\partial \omega_{j,k}^\ell} = \omega_{j,k}^L - \frac{\eta}{n}(\hat{y}_j - y_j)h_k^L$$

For future use, we refer to the increment used to update $\omega_{j,k}^L$ as

$$\delta_{j,k}^L = \frac{1}{n}(\hat{y}_j - y_j)h_k^{L-1}.$$

Notice that this includes the division by n (the total number of training examples) but not the learning rate η. Many treatments leave out the factor of n^{-1} since we end up multiplying the entire derivative by an arbitrary learning rate anyway. NOTE: Once again, updating the weights should only be done at the very end of the process! Otherwise, the computation of the gradient will not be correct.

Consider partial derivatives for the next layer down. The weight $\omega_{j,k}^{L-1}$ appears in the computation of h_j^L. But h_j^L is used in the computation of every output $\hat{y}_p$!

$$\begin{aligned}
\frac{1}{n}\frac{\partial L}{\partial \omega_{j,k}^{L-1}} &= \frac{1}{n}\frac{\partial}{\partial \omega_{j,k}^{L-1}}\sum_{p=1}^{m}\left[-y_p\ln(\hat{y}_p)-(1-y_p)\ln(1-\hat{y}_p)\right] \\
&= \frac{1}{n}\sum_{p=1}^{m}\frac{\hat{y}_p-y_p}{\hat{y}_p(1-\hat{y}_p)}\frac{\partial \hat{y}_p}{\partial \omega_{j,k}^{L-1}} \\
&= \frac{1}{n}\sum_{p=1}^{m}\frac{\hat{y}_p-y_p}{\hat{y}_p(1-\hat{y}_p)}\hat{y}_p(1-\hat{y}_p)\omega_{p,j}^{L}\frac{\partial h_j^{L-1}}{\partial \omega_{j,k}^{L-1}} \\
&= \frac{1}{n}\sum_{p=1}^{m}(\hat{y}_p-y_p)\omega_{p,j}^{L}h_j^{L-1}(1-h_j^{L-1})h_k^{L-2} \\
&= \left(\sum_{p=1}^{m}\delta_{p,j}^{L}\omega_{p,j}^{L}\right)(1-h_j^{L-1})h_k^{L-2}
\end{aligned}$$

This gives us

$$\omega_{j,k}^{L-1} \longleftarrow \omega_{j,k}^{L-1}-\eta\delta_{j,k}^{L-1}$$

where

$$\delta_{j,k}^{L-1} = \left(\sum_{p=1}^{m}\delta_{p,j}^{L}\omega_{p,j}^{L}\right)(1-h_j^{L-1})h_k^{L-2}.$$

Again, updates should only be done at the end of the process since the current weights are needed in the next layer down.

If we look at the updates for one more layer down, the general pattern

should become clear.

$$
\begin{aligned}
\frac{1}{n}\frac{\partial C}{\partial \omega_{j,k}^{L-2}} &= \frac{\partial}{\partial \omega_{j,k}^{L-2}} \sum_{p=1}^{m} \left[-y_p \ln(\hat{y}_p) - (1-y_p)\ln(1-\hat{y}_p)\right] \\
&= \frac{1}{n}\sum_{p=1}^{m} \frac{\hat{y}_p - y_p}{\hat{y}_p(1-\hat{y}_p)} \frac{\partial \hat{y}_p}{\partial \omega_{j,k}^{L-2}} \\
&= \frac{1}{n}\sum_{p=1}^{m} \frac{\hat{y}_p - y_p}{\hat{y}_p(1-\hat{y}_p)} \hat{y}_p(1-\hat{y}_p) \sum_{q=1}^{n_L} \omega_{p,q}^{L} \frac{\partial h_q^{L-1}}{\partial \omega_{j,k}^{L-2}} \\
&= \frac{1}{n}\sum_{p=1}^{m}\sum_{q=1}^{n_L} (\hat{y}_p - y_p)\omega_{p,q}^{L} h_q^{L-1}(1-h_q^{L-1})\omega_{q,j}^{L-1} \frac{\partial h_j^{L-2}}{\partial \omega_{j,k}^{L-2}} \\
&= \frac{1}{n}\sum_{p=1}^{m}\sum_{q=1}^{n_L} (\hat{y}_p - y_p) h_q^{L-1}\omega_{p,q}^{L}(1-h_q^{L-1}) h_j^{L-2}(1-h_j^{L-2})\omega_{q,j}^{L-1} h_k^{L-3} \\
&= \sum_{q=1}^{n_L}\left(\sum_{p=1}^{m} \delta_{p,q}^{L}\omega_{p,q}^{L}\right)(1-h_q^{L-1}) h_j^{L-2}(1-h_j^{L-2})\omega_{q,j}^{L-1} h_k^{L-3} \\
&= \left(\sum_{q=1}^{n_L} \delta_{q,j}^{L-1}\omega_{q,j}^{L-1}\right)(1-h_j^{L-2}) h_k^{L-3}
\end{aligned}
$$

Defining the increment

$$
\delta_{j,k}^{L-2} = \left(\sum_{q=1}^{n_L} \delta_{q,j}^{L-1}\omega_{q,j}^{L-1}\right)(1-h_j^{L-2}) h_k^{L-3},
$$

we have

$$
\omega_{j,k}^{L-2} \longleftarrow \omega_{j,k}^{L-2} - \eta\delta_{j,k}^{L-2}.
$$

Our work above suggests suggests a general pattern for $1 \le \ell \le L$.

$$
\begin{aligned}
\delta_{j,k}^{\ell-1} &= \left(\sum_{p=1}^{n_\ell} \delta_{p,j}^{\ell}\omega_{p,j}^{\ell}\right)(1-h_j^{\ell-1}) h_k^{\ell-2} \\
\omega_{j,k}^{\ell-1} &\longleftarrow \omega_{j,k}^{\ell-1} + \eta\delta_{j,k}^{\ell-1}
\end{aligned}
$$

This gives us a way to program backprop for our neural net with logistic sigmoid activation function.

0) **Initialization:** Randomly choose weights $\left\{\omega_{i,j}^{\ell}\right\}$ for all layers and neurons (e.g. draw an appropriate number of samples from Uniform(-1,1)). Set $C_{\text{old}} = 0$.

1) **Backward Propagation:** For each of the n training samples $(\boldsymbol{x}_i, \boldsymbol{y}_i)$, forward propagate the data $\boldsymbol{x}_i$ from input to the output layers to obtain $\hat{\boldsymbol{y}}_i$. Store all intermediate layer values $\boldsymbol{h}_i^{\ell}$.

   a) Compute (and store) the increments for the uppermost layer.
   For $1 \leq j \leq m$ and $1 \leq k \leq n_L$:

$$\delta_{j,k}^{L} = \frac{1}{n}(\hat{y}_{i,j} - y_{i,j})h_{i,k}^{L-1}.$$

   b) Compute (and store) the increments for the intermediate layers.
   For $2 \leq \ell \leq L-1$, $1 \leq j \leq n_\ell$ and $1 \leq k \leq n_{\ell-1}$:

$$\delta_{j,k}^{\ell-1} = \left(\sum_{p=1}^{n_\ell} \delta_{p,j}^{\ell}\omega_{p,j}^{\ell}\right)(1 - h_{i,j}^{\ell-1})h_{i,k}^{\ell-2}.$$

   c) Compute the increments for the lowest layer.
   For $1 \leq j \leq n_0$ and $1 \leq k \leq n$:

$$\delta_{j,k}^{0} = \left(\sum_{p=1}^{n_1} \delta_{p,j}^{1}\omega_{p,j}^{1}\right)(1 - h_{i,j}^{0})x_{i,k}.$$

   d) Update all weights.
   For all weights,

$$\omega_{j,k}^{\ell} \longleftarrow \omega_{j,k}^{\ell} - \eta\delta_{j,k}^{\ell}.$$

2) **Threshold Testing:** Compute the total cost for all training samples $C(\hat{\boldsymbol{y}}, \boldsymbol{y})$ with the updated weights. Compute $\Delta C = |C(\hat{\boldsymbol{y}}, \boldsymbol{y}) - C_{\text{old}}|$. If $\Delta C < \epsilon$, then quit. Otherwise, set $C_{\text{old}} = C(\hat{\boldsymbol{y}}, \boldsymbol{y})$.

3) **Iteration:** Repeat steps 1) and 2) until the condition in 2) is met.

4.4 General Backpropagation

Our considerations above generalize to any type of loss function:

$$C(\hat{\boldsymbol{y}}, \boldsymbol{y}) = \frac{1}{n} \sum_{i=1}^{n} L_i(\hat{\boldsymbol{y}}_i, \boldsymbol{y}_i)$$

where in general we could have different losses for different data points (though in many applications the loss function is identical for all data points). We can also allow any kind of non-linear activation function

$$h_{i,j}^{\ell} = a\left(\boldsymbol{\theta}_j^{\ell} \cdot \boldsymbol{h}_i^{\ell-1}\right) = a\left(\omega_{j,1}^{\ell} h_{i,1}^{\ell-1} + \omega_{j,2}^{\ell} h_{i,2}^{\ell-1} + \cdots + \omega_{j,n_{\ell-1}}^{\ell} h_{i,n_{\ell-1}}^{\ell-1}\right)$$

where $a = a(z)$ is the chosen activation.

Training a neural network requires the repetition of three major steps which must be iterated over each individual data point for some number of rounds. Each single run through all data points is usually referred to as an *epoch*.

1) **Forward propagation:** use input $\boldsymbol{x}_i$ to compute $h_{i,j}^{\ell}$ at every layer.

2) **Backward propagation:** compute the negative gradient of the loss function for the current set of weights.

3) **Update:** use the negative gradient to update the weights.

Forward propagation is straightforward given the neural network. Backpropagation generalizes to the following steps. Note that the $\tilde{\delta}$ terms are intermediate values used to compute the actual partial derivatives in the next layer down and that the weights are only updated once all gradient terms are computed.

For each data point $\boldsymbol{x}_i$:

a) Compute (and store) the increments for the uppermost layer.

   For $1 \leq j \leq m$ and $1 \leq k \leq n_L$:

$$\tilde{\delta}_{j,k}^{L} = \frac{\partial L_i}{\partial \hat{y}_{i,j}} \cdot a'\left(\boldsymbol{\theta}_j^{L} \cdot \boldsymbol{h}_i^{L-1}\right),$$
$$\delta_{j,k}^{L} = \tilde{\delta}_{j,k}^{L} h_{i,k}^{L-1}.$$

b) Compute (and store) the increments for the intermediate layers.

For $2 \leq \ell \leq L-1$, $1 \leq j \leq n_\ell$ and $1 \leq k \leq n_{\ell-1}$:

$$\tilde{\delta}_{j,k}^{\ell-1} = a'\left(\boldsymbol{\theta}_j^{\ell-1} \cdot \boldsymbol{h}_i^{\ell-2}\right) \sum_{p=1}^{n_\ell} \tilde{\delta}_{p,j}^{\ell} \omega_{p,j}^{\ell},$$

$$\delta_{j,k}^{\ell-1} = \tilde{\delta}_{j,k}^{\ell-1} h_{i,k}^{\ell-2}.$$

c) Compute the increments for the lowest layer.

For $1 \leq j \leq n_0$ and $1 \leq k \leq n$:

$$\tilde{\delta}_{j,k}^{0} = a'\left(\boldsymbol{\theta}_j^{0} \cdot \boldsymbol{x}_i\right) \sum_{p=1}^{n_\ell} \tilde{\delta}_{p,j}^{1} \omega_{p,j}^{1},$$

$$\delta_{j,k}^{0} = \tilde{\delta}_{j,k}^{0} x_{i,k}.$$

d) Update the weights.

For all weights,

$$\omega_{j,k}^{\ell} \longleftarrow \omega_{j,k}^{\ell} - \eta \delta_{j,k}^{\ell}.$$

5 Lectures 5 and 6: Support Vector Machines

February 20 and 26, 2024

5.1 Introduction to Support Vector Machines (SVMs)

In its most basic version, a support vector machine (SVM) attempts to learn a hyperplane that effectively separates points in a data set into two distinct groups. As usual, we assume that our dataset as N examples. Each example has d features and a binary class label. For SVMs, it is convenient to use $+1$ for the positive class and -1 for the negative class. We use $\{\mathbf{x}_i\}_{i=1}^N$ for the set of features where

$$\mathbf{x}_i = \begin{bmatrix} x_{i,1} \\ x_{i,2} \\ \vdots \\ x_{i,d} \end{bmatrix}.$$

The class labels are $y_i = \pm 1$.

In any number of dimensions, a plane is specified by a vector normal to the plane, denoted $\mathbf{w}$, and an offset (or bias), b. Points on the plane satisfy

$$\mathbf{w} \cdot \mathbf{x} + b = \mathbf{w}^T \mathbf{x} + b = 0.$$

The idea is to identify an optimal normal vector $\mathbf{w}$ and bias b so that the function

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

effectively separates the points from different classes. That is, we want $f(\mathbf{x}) > 0$ for examples in the positive class and $f(\mathbf{x}) < 0$ for examples in the negative class.

The hyperplane identified by this process serves as a *decision boundary* between the two classes. As it turns out, SVMs can be modified to find non-linear decision boundaries as well! We will begin by examining the linear case in detail. After that, we will discuss the ways in which non-linear decision boundaries can be incorporated.

5.2 Linear SVMs

Based on the description above, our problem is to find an efficient *linear* separator for our binary data. A simple cartoon of the situation is shown below (red dots are from the negative class and green from the positive).

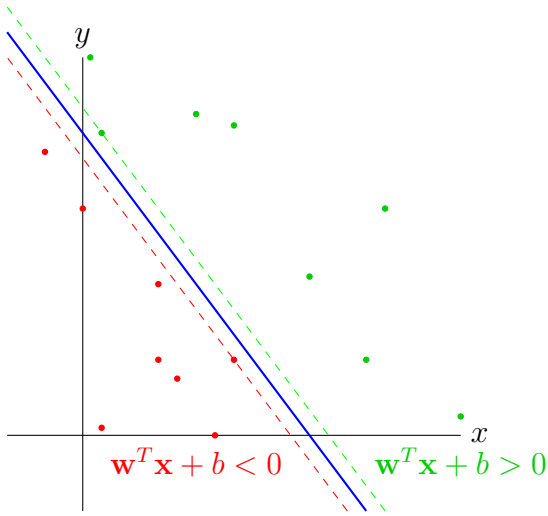


Figure 11: Linear SVM in 2D

As we can see from the diagram, the point (or points) in either class that are closest to the dividing hyperplane play an important role. These special data points are known as the *support vectors*. Notice that any hyperplane in the channel between the support vectors would make an effective decision boundary, but we choose the one that is equidistant from the support vectors in either class.

As is well known, the distance between a point $\tilde{\mathbf{x}}$ and hyperplane $\mathbf{w}^T\mathbf{x} + b = 0$ is given by

$$d = \frac{|\mathbf{w}^T\tilde{\mathbf{x}} + b|}{\|\mathbf{w}\|}.$$

Suppose that $\mathbf{x}_+$ is a support vector from the positive category and $\mathbf{x}_-$ is a support vector from the negative category. Since the dividing hyperplane is equidistant from this line, we have

$$\mathbf{w}^T\mathbf{x}_+ + b = a,$$
$$\mathbf{w}^T\mathbf{x}_- + b = -a$$

for some positive constant a. By rescaling the normal vector $\mathbf{w}$, we can assume that $a = 1$. This means that distance from the hyperplane to any support vector is

$$d_\pm = \frac{|\mathbf{w}^T\mathbf{x}_\pm + b|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}$$

and the width of the channel between the two classes is $\frac{2}{\|\mathbf{w}\|}$. So, we need to find the hyperplane that maximizes this distance between the two classes.

Clearly, maximizing $\frac{2}{\|\mathbf{w}\|}$ is equivalent to minimizing $\|\mathbf{w}\|$. Even better, we can minimize $\|\mathbf{w}\|$ by minimizing $\frac{1}{2}\|\mathbf{w}\|^2$ (the factor of one–half is merely for convenience when taking derivatives). Our constraints are $\mathbf{w}^T\mathbf{x_i} + b \geq 1$ for members of the positive class while $\mathbf{w}^T\mathbf{x_i} + b \leq 1$ for members of the negative class. Since $y_i = 1$ on the positive class and $y_i = -1$ on the negative class. we can combine these constraints to

$$y_i\left(\mathbf{w}^T\mathbf{x_i} + b\right) \geq 1 \qquad \text{for all } i.$$

We are lead to the following minimization problem.

$$\begin{aligned} \text{Minimize: } & \frac{1}{2}\|\mathbf{w}\|^2 \\ \text{subject to: } & y_i\left(\mathbf{w}^T\mathbf{x_i} + b\right) - 1 \geq 0 \qquad (i = 1, 2, \cdots N). \end{aligned}$$

As we noted, the support vectors should play the most important role, and for these special vectors, $y_i\left(\mathbf{w}^T\mathbf{x_i} + b\right) - 1 = 0$. We will use this special form of the constraint for the next stage.

Using Lagrange Multipliers, we can reformulate this problem as an unconstrained minimization of the Lagrangian function

$$L(\mathbf{w}, \boldsymbol{\alpha}, b) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{N}\left\{\alpha_i y_i\left(\mathbf{w}^T\mathbf{x_i} + b\right) - \alpha_i\right\}.$$

Taking the gradient with respect to $\mathbf{w}$ and setting this quantity equal to 0 gives the following.

$$\begin{aligned} \nabla_{\mathbf{w}} L &= \mathbf{w} - \sum_{i=1}^{N}\alpha_i y_i \mathbf{x_i} = 0 \\ \mathbf{w} &= \sum_{i=1}^{N}\alpha_i y_i \mathbf{x_i}. \end{aligned}$$

Taking the partial derivative with respect to b gives another important result.

$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{N} \alpha_i y_i = 0$$
$$\sum_{i=1}^{N} \alpha_i y_i = 0$$

The gradient with respect to $\boldsymbol{\alpha}$ gives us back the constraints.

We can use the duality of quadratic programming problems to recast our minimization problem as the following maximization problem.

$$\text{Maximize: } \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j \mathbf{x_i}^T \mathbf{x_j}$$
$$\text{subject to: } \sum_{i=1}^{N} \alpha_i y_i = 0 \text{ and } \alpha_i \geq 0.$$

Intuitively since the α_i are positive, the maximum will hinge on making the quantity in the second term as negative as possible.

Consider the simplest case where $\alpha_i = 0, 1$. If $\mathbf{x_i}$ and $\mathbf{x_j}$ are nearly orthogonal, then $\mathbf{x_i}^T \mathbf{x_j} \approx 0$ and this pair will not contribute much to the overall total. Assuming that the features are normalized, vectors that are roughly in the same direction will have $\mathbf{x_i}^T \mathbf{x_j} \approx 1$. If these features belong to the same class, then $y_i y_j = 1$ and we have a positive term which will make the overall sum smaller. So, pairs of nearly parallel vectors in the same class will be downgraded in the maximization. On the other hand, if the examples are from different classes then $y_i y_j = -1$ and the term will make the overall total larger. Hence, the maximization process will want to upgrade pairs that are nearly parallel but from different classes. Finally, if the features are anti–parallel, $\mathbf{x_i}^T \mathbf{x_j} \approx -1$ and training samples like this from the same class will increase the total sum. However, the constraint forces us to have the same number of non-zero α_i from each class. So anti–parallel samples from the same class are not favored. Hence, the maximization process will select pairs (i, j) where $\mathbf{x_i}$ and $\mathbf{x_j}$ are nearly parallel but from opposite classes.

As argued above, the α_i which are positive will correspond to the support vectors – training samples from each class that are closest to the decision boundary. After solving the maximization problem, we compute the normal

vector by

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i y_i \mathbf{x_i}.$$

The bias b can be determined from

$$b = y_k - \mathbf{w}^T \mathbf{x_k}$$

where k is any training sample with $\alpha_k > 0$ (that is k is a support vector).

One process for finding the optimal Lagrange multipliers $\boldsymbol{\alpha}$ and bias b is the Sequential Minimal Optimization (SMO) algorithm. It can be shown that the following Karush–Kuhn–Tucker (KKT) conditions must be satisfied in order for a solution to be optimal.

$$\begin{aligned} \alpha_i = 0 &\longrightarrow y_i \left(\mathbf{w}^T \mathbf{x_i} + b\right) \geq 1 \\ \alpha_i = C &\longrightarrow y_i \left(\mathbf{w}^T \mathbf{x_i} + b\right) \leq 1 \\ 0 < \alpha_i < C &\longrightarrow y_i \left(\mathbf{w}^T \mathbf{x_i} + b\right) = 1 \end{aligned}$$

Here C is a positive parameter that essentially sets the penalty for any misclassified instances. The basic idea of the SMO Algorithm is to first find an index i so that α_i does not satisfy these KKT conditions. Having selected i, we then pick a companion j from the remaining indices and optimize the pair α_i and α_j. This process continues until all α_i satisfy the KKT conditions. In order to speed up the algorithm, heuristics are used to pick an optimal companion index (since the number of pairs is $O(N^2)$).

5.3 Generalization to Non–Linear SVMs

In order to incorporate non–linear effects into SVMs, we can replace $\mathbf{x_i}$ with a mapping $\phi(\mathbf{x_i})$. This turns our decision function into

$$f(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x}) + b.$$

From our discussion above, we know that

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i y_i \phi(\mathbf{x_i})$$

which means that

$$f(\mathbf{x}) = \sum_{i=1}^{N} \alpha_i y_i \phi(\mathbf{x_i})^T \phi(\mathbf{x}) + b.$$

If we denote the non–linear kernel by $K(\mathbf{x_1}, \mathbf{x_2}) = \phi(\mathbf{x_1})^T \phi(\mathbf{x_2})$ then we have

$$f(\mathbf{x}) = \sum_{i=1}^{N} \alpha_i y_i K(\mathbf{x_i}, \mathbf{x}) + b.$$

Tracing through our arguments before leads to the following maximization problem.

$$\begin{aligned} \text{Maximize: } & \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j K(\mathbf{x_i}, \mathbf{x_j}) \\ \text{subject to: } & \sum_{i=1}^{N} \alpha_i y_i = 0 \text{ and } \alpha_i \geq 0 \end{aligned}$$

which can be solved with the SMO Algorithm. Common choices for the kernel are listed below.

Name	Kernel
Linear:	$K(\mathbf{x_1}, \mathbf{x_2}) = \mathbf{x_1}^T \mathbf{x_2}$
Polynomial:	$K(\mathbf{x_1}, \mathbf{x_2}) = \left(\mathbf{x_1}^T \mathbf{x_2} + 1\right)^d$
Gaussian:	$K(\mathbf{x_1}, \mathbf{x_2}) = \exp\left(-\gamma \lVert \mathbf{x_1} - \mathbf{x_2} \rVert^2\right)$
Sigmoid:	$K(\mathbf{x_1}, \mathbf{x_2}) = \tanh\left(\gamma \mathbf{x_1}^T \mathbf{x_2} + \eta\right)$

Figure 12: Common Kernel Functions for SVMs

6 Lectures 7 and 8: Unsupervised Learning Techniques

February 26 and and March 18, 2024

6.1 k–Means Clustering

The goal of the k–Means Algorithm is to group data points into a number of clusters that appear close together as measured by some metric. The image

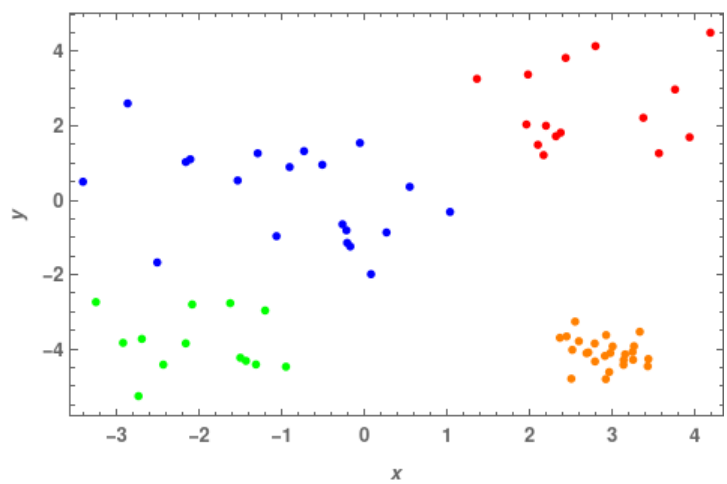


Figure 13: Two Dimensional Clustering Example

above has been colored to make it clear that there are 4 distinct clusters. However, the number of clusters is a parameter that must be set before the algorithm can be run. If the data is easy to represent graphically, then the number of clusters may be obvious. However, the optimum choice for the number of clusters is usually not easy to see in advance. More on this later.

Suppose we have selected an integer $k \geq 2$ for the number of clusters. The k–Means algorithm proceeds as follows.

1) Randomly choose k initial centroids in the domain of the data.

2) Assign each data point to the centroid that is closest to it.

3) Compute new centroids for each of the clusters from 2).

4) Repeat steps 2) and 3) until the centroids stabilize within some tolerance (or a maximum number of iterations has been reached).

Once the final centroids have been found, classification is easy. Any new data point is classified to the cluster belonging to the centroid it is closest to.

The only real difficulty in the method is choosing a decent value for the number of centroids. If k is chosen too small, we will get overly broad categories which are unlikely to distinguish genuinely different types of data. On the other hand, having too many centroids will lead to a huge number of small clusters – greatly limiting the explanatory power of the model. One popular heuristic for determining an optimal range for k is the *elbow method*.

Instead of choosing k upfront, we run the k–Means Algorithm for a range of k. For each run, we compute some metric designed to evaluate the goodness of fit. One such measure is the mean squared error for each cluster (or its square root). A typical plot of this is shown below. As we can see, there is

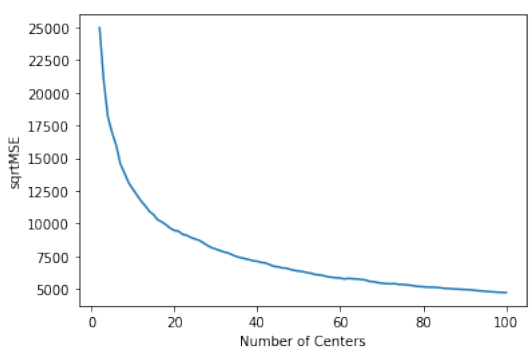


Figure 14: A Typical "Elbow" Plot

a region where the overall improvement in the error seems to level off. The elbow method advises us to choose the number of clusters in this turning region (the "elbow" of the graph). In the sample above, the elbow region appears to be around 20 clusters.

While mean squared error is a decent measure for goodness–of–fit for k–Means, there are others worth exploring. One important alternate metric is the Davies–Bouldin Index. Two begin, suppose we have two distinct clusters labeled i and j with centroids $\mathbf{c_i}$ and $\mathbf{c_j}$, respectively. Suppose that cluster i contains data points $\{\mathbf{x_{i,\ell}}\}$ for $1 \leq \ell \leq n_i$ where n_i is the total number of data points in the cluster (*mutatis mutandis* for cluster j). The variance for

each clusters is computed by

$$\mathrm{Var}(\mathbf{c_i}) = \frac{1}{n_i} \sum_{\ell=1}^{n_i} \|\mathbf{c_i} - \mathbf{x_{i,\ell}}\|^2.$$

The variance is a natural measure for how tightly grouped a cluster is. In order to measure the separation of clusters, we use the distance between their centroids ($\|\mathbf{c_i} - \mathbf{c_j}\|$). For each pair of clusters, we can define a separation score

$$R_{ij} = \frac{\mathrm{Var}(\mathbf{c_i}) + \mathrm{Var}(\mathbf{c_j})}{\|\mathbf{c_i} - \mathbf{c_j}\|}.$$

Note that if the two clusters are fairly tight and reasonably far from each other, then R_{ij} will be small (clearly $R_{ij} = R_{ji}$). On the other hand, if one of the clusters is fairly diffuse or if the two clusters are close together, the score will be large.

For each cluster, we define an index D_i by

$$D_i = \max_{j \neq i} \{R_{ij}\}.$$

The Davies–Bouldin Index for the entire set of clusters is then given by

$$\mathrm{DB} = \frac{1}{k} \sum_{i=1}^{k} D_i.$$

The advantage of this index over simple mean squared error is that it accounts for both the tightness of clusters and their overall separation from one another.

As with total mean squared error for the clusters, the Davies–Bouldin index will eventually go to zero. To understand why, imagine that all distinct data points are made the centroid of their own cluster. In that extreme case, the cluster variances will all be zero while the centroids are all distinct. Hence, $R_{ij} = 0$ for all pairs $i \neq j$ and $\mathrm{DB} = 0$. Unlike total mean squared error, the Davies–Bouldin Index will not monotonically decrease to zero. The plot below shows the Davies–Bouldin Index for the same set of data as the elbow plot above. The Davies–Bouldin Index seems to suggest choosing 35 to 40 clusters in this case (fairly larger than the 20 or so clusters from the elbow method), but the jaggedness of the plot makes it much harder to draw firm conclusions.

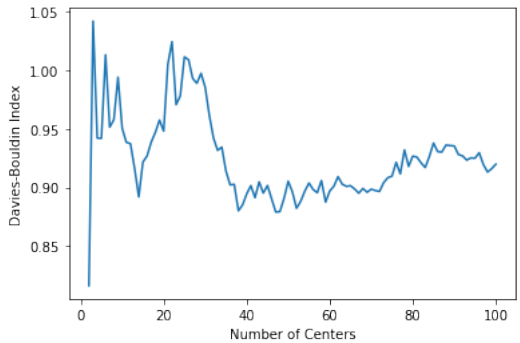


Figure 15: A Plot of Davies–Bouldin Index

6.2 Hierarchical Clustering

As mentioned above, the main issue with k–Means is selecting an appropriate value for the total number of clusters. Hierarchical Clustering delays this decision until after the algorithm is completed. There are two basic types of Hierarchical Clustering methods.

1) **Agglomerative Methods:** Data points start as individual clusters. At each stage, points are merged into clusters, and the process continues until there is a single cluster.

2) **Divisive Methods:** All data points start in the same cluster. At each stage, clusters are split, and the process continues until only singleton clusters remain.

Agglomerative techniques are far more common and will be the focus below. In either case, the level of clustering can be chosen after the algorithm has been completed.

In order to merge clusters, we need to decide on some measure of their proximity to one another. At each stage of the clustering algorithm, we compute the proximity measure for each pair of clusters and merge the two clusters which are closest according to that measure. Once there is only one cluster, the algorithm terminates. Common proximity measures are shown below.

For visualization purposes, a *dendrogram* can often be used to understand how clusters were formed during the process. Such a diagram consists of individual data points placed on a line. As clusters are merged, a link between

Name	Description
MIN	Smallest pairwise distance between points from different clusters
MAX	Largest pairwise distance between points from different clusters
AVG	Average distance between pairs of points from different clusters

Figure 16: Common Proximity Measures for Hierarchical Clustering

the clusters is added (with the height indicating the step at which the merger took place). The dendrogram can make it relatively easy to spot a decent level of clustering for the problem.

The example dendrogram below is for the set $\{10, 12, 20, 21, 40, 45, 98\}$. Each cluster is marked by its centroid, and distance between the centroids is used to merge the clusters.

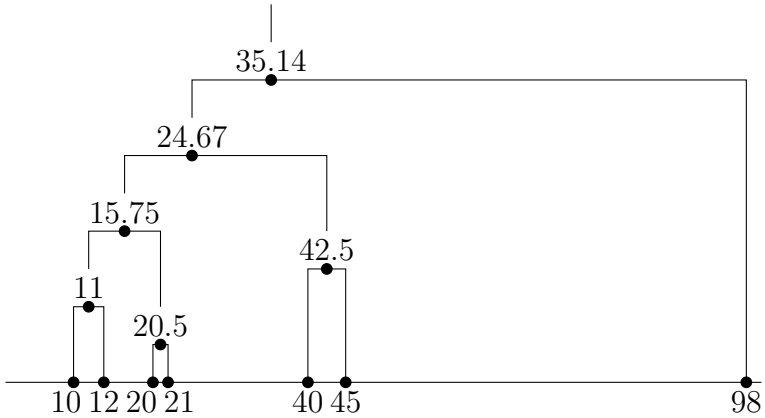


Figure 17: A Sample Dendrogram

6.3 DBSCAN

DBSCAN is a density–based algorithm for identifying clusters. Unlike the previous methods, DBSCAN can readily identify clusters of arbitrary shape and sizes. Unfortunately, it can be expensive to implement and often fails in very high dimensions or if clusters have wildly different densities.

To begin, the user must set two parameters: ϵ (which sets the distance scale) and `MinPts` (which effectively sets the density threshold). DBSCAN also requires a distance metric between data points. As with k–Means, the

method can be run with different choices for these parameters to determine which combination works best for the particular data being studied.

Once the parameters above have been chosen, data points can be divided into three categories.

1) **Core Points:** Data points that have at least `MinPts` other data points within a distance of ϵ

2) **Border Points:** Data points that are not core points but are within a distance of ϵ of at least one core point

3) **Noise Points:** Data points that are neither core nor border points

The DBSCAN algorithm first classifies each data point as Core, Border, or Noise and discards the Noise Points. It then connects each Core point to any other Core Point which is within a distance of ϵ. Connected groups of Core Points are identified as clusters, and the algorithm finishes by assigning Border Points to their closest cluster. The diagram below shows an application of DBSCAN in two–dimensions. There are three clusters (shown in green, blue, and orange) and a number of noise points (shown in gray).

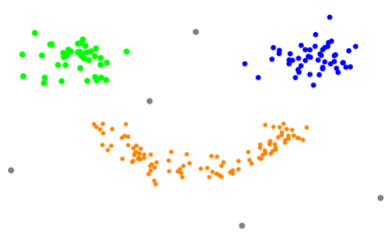


Figure 18: DBSCAN Clustering in 2D

6.4 Fuzzy Clustering

If the data can be partitioned into well–separated groups, then traditional k–Means Clustering is a good choice. Often however, data points do not clearly belong to one group or another. In such a case, we can relax the idea of membership in a cluster from strictly binary (0 = excluded or 1 = included) to probabilistic. Allowing objects to belong to various sets with

some probability is called *fuzzy set theory* or *fuzzy logic*.[1] Here, we will consider the Fuzzy k–Means algorithm which extends the traditional k–Means algorithm to fuzzy clusters.

As before, suppose we decide to have k clusters with centroids $\mathbf{c_j}$. As we will see, we do not need to choose random centroids up front. For each data point $\mathbf{x}_i$ $(1 \leq i \leq n)$, we assign a set of initial (randomly chosen) probabilities p_{ij} which give us the level of membership for this data point in each of the clusters. Since these are probabilities, we must have

$$\sum_{j=1}^{k} p_{ij} = 1$$

for each data point $\mathbf{x}_i$. Apart from the number of clusters, there is an additional hyperparameter which must be chosen for this model: m determines the *degree of fuzziness* of the clusters as we will see below. Note that we must have $m > 1$.

Since every data point potentially has some level of membership in every cluster, we no longer determine which centroid is closest to the data points in order to update the clusters. Rather, we directly update the clusters using the probabilities.

$$\mathbf{c_j} = \frac{\sum_{i=1}^{n} p_{ij}^m \mathbf{x}_i}{\sum_{i=1}^{n} p_{ij}^m}$$

Note the exponent of m on all of the probabilities. Once we update the centroids, we then update the prbabilities by

$$p_{ij} = \frac{\frac{1}{\|\mathbf{x}_i - \mathbf{c_j}\|^{2/(m-1)}}}{\sum_{\ell=1}^{k} \frac{1}{\|\mathbf{x}_i - \boldsymbol{c_\ell}\|^{2/(m-1)}}} = \frac{1}{\sum_{\ell=1}^{k} \left(\frac{\|\mathbf{x}_i - \mathbf{c_j}\|}{\|\mathbf{x}_i - \boldsymbol{c_\ell}\|}\right)^{2/(m-1)}}. \tag{5}$$

Note that the first form makes it clear that these probabilities sum up to

[1] The use of "probability" is generally frowned upon in the fuzzy logic community. The weights are usually referred to as "membership degrees."

one.

$$\begin{aligned}
\sum_{j=1}^{k} p_{ij} &= \sum_{j=1}^{k} \frac{\frac{1}{\|\mathbf{x}_i - \mathbf{c_j}\|^{2/(m-1)}}}{\sum_{\ell=1}^{k} \frac{1}{\|\mathbf{x}_i - \boldsymbol{c_\ell}\|^{2/(m-1)}}} \\
&= \left(\frac{1}{\sum_{\ell=1}^{k} \frac{1}{\|\mathbf{x}_i - \boldsymbol{c_\ell}\|^{2/(m-1)}}} \right) \left(\sum_{j=1}^{k} \frac{1}{\|\mathbf{x}_i - \mathbf{c_j}\|^{2/(m-1)}} \right) \\
&= \left(\frac{1}{\sum_{\ell=1}^{k} \frac{1}{\|\mathbf{x}_i - \boldsymbol{c_\ell}\|^{2/(m-1)}}} \right) \left(\sum_{\ell=1}^{k} \frac{1}{\|\mathbf{x}_i - \mathbf{c}_\ell\|^{2/(m-1)}} \right) = 1
\end{aligned}$$

We repeat these two updates until the centroids stabilize within a specified tolerance (or until a threshold number of iterations has been reached). Once we have determined the centroids during the training phase, new data points can be assigned probabilities using (5). As with regular k–Means, it can be shown that this procedure attempts to minimize the fuzzy sum of squared error.

$$\text{fuzzy SSE} = \sum_{j=1}^{k} \sum_{i=1}^{n} p_{ij}^{m} \|\mathbf{x}_i - \mathbf{c_j}\|^2.$$

To conclude, we should examine the role of m in some detail. First consider the sum

$$\sum_{\ell=1}^{k} \left(\frac{\|\mathbf{x}_i - \mathbf{c_j}\|}{\|\mathbf{x}_i - \boldsymbol{c_\ell}\|} \right)^{2/(m-1)}$$

appearing in the denominator of (5). We first examine the limit $m \to 1^+$ (that is, we allow m to get close to one from values larger than one).

Suppose that cluster r is the closest centroid to $\mathbf{x}_i$.[2] Then the largest term in the sum is

$$\left(\frac{\|\mathbf{x}_i - \mathbf{c_j}\|}{\|\mathbf{x}_i - \boldsymbol{c_r}\|} \right)^{2/(m-1)}$$

which is definitely greater than or equal to one. If $i \neq r$ (and assuming all centroids are unique), this quantity is definitely bigger than one. That means

$$\lim_{m \to 1^+} \left(\frac{\|\mathbf{x}_i - \mathbf{c_j}\|}{\|\mathbf{x}_i - \boldsymbol{c_r}\|} \right)^{2/(m-1)} = \infty$$

[2]In the unlikely even that a data point is identical to centroid $\boldsymbol{c_r}$ we take $p_{ir} = 1$ and all other $p_{ij} = 0$.

since the exponent is also approaching positive infinity. Hence, when j is not the closest cluster to $\boldsymbol{x}_i$ we must have

$$\lim_{m \to 1^+} p_{ij} = 0.$$

On the other hand, when $j = r$ we have

$$\left(\frac{\|\mathbf{x}_i - \mathbf{c_r}\|}{\|\mathbf{x}_i - \boldsymbol{c_r}\|} \right)^{2/(m-1)} = 1$$

regardless of m. But since the remaining terms are all less than one, we have

$$\lim_{m \to 1^+} \left(\frac{\|\mathbf{x}_i - \mathbf{c_r}\|}{\|\mathbf{x}_i - \boldsymbol{c_\ell}\|} \right)^{2/(m-1)} = 0$$

whenever $\ell \neq r$. That means

$$\lim_{m \to 1^+} p_{ir} = 1.$$

To summarize, as m approaches 1 the Fuzzy k–Means algorithm reverts to traditional k–Means (where each data point is definitely assigned to the cluster with closest centroid).

Now consider what happens as we allow $m \to \infty$.

$$\lim_{m \to \infty} \frac{2}{m-1} = 0$$

So, we have

$$\lim_{m \to \infty} \left(\frac{\|\mathbf{x}_i - \mathbf{c_j}\|}{\|\mathbf{x}_i - \boldsymbol{c_\ell}\|} \right)^{2/(m-1)} = 1$$

for any choice of the indices (assuming none of the values is zero). But that means

$$\lim_{m \to 1^+} p_{ij} = \frac{1}{k}$$

and each data point has equal membership in all of the clusters! In other words, the larger we make m, the fuzzier the clusters will be. A common choice for the degree of fuzziness is $m = 2$.

7 Lecture 9: Expectation Maximization

April 8, 2024

7.1 Bayes' Formula

Recall from basic probability theory that the conditional probability of an event E given that some event F has occurred is defined to be

$$\mathbb{P}(E|F) = \frac{\mathbb{P}(E \cap F)}{\mathbb{P}(F)}.$$

Clearly, event F must have non–zero probability for this to be sensible.

As a direct consequence of this definition, we have Bayes' Formula.

$$\begin{aligned}\mathbb{P}(E|F) &= \frac{\mathbb{P}(E \cap F)}{\mathbb{P}(F)} \\ &= \frac{\mathbb{P}(E \cap F)}{\mathbb{P}(E)} \frac{\mathbb{P}(E)}{\mathbb{P}(F)} \\ &= \frac{\mathbb{P}(F|E)\mathbb{P}(E)}{\mathbb{P}(F)}\end{aligned}$$

As above, both events E and F must have non–zero probability for this to make sense.

Quite often, this formula is used to update probabilities given some partial knowledge about the outcome. The idea is that we set some *prior* probability distribution on event E before we have any direct knowledge of outcomes. Once the information encoded in event F is made known to us, we update our assessment of the probability of event E to the *posterior* probability distribution $\mathbb{P}(E|F)$.

A typical usage is where we wish to estimate the probabilities associated to certain model parameters collectively referred to as $\boldsymbol{\theta} = (\theta_1, \theta_2, \cdots \theta_n)$. Once we collect data, we would like to assess the likelihood that these parameters take on particular values.

$$\mathbb{P}(\boldsymbol{\theta} \mid \text{ Data }) = \frac{\mathbb{P}(\text{ Data } \mid \boldsymbol{\theta})\mathbb{P}(\boldsymbol{\theta})}{\mathbb{P}(\text{ Data })}$$

$\mathbb{P}(\text{ Data } \mid \boldsymbol{\theta})$ is frequently a well–known probability distribution. $\mathbb{P}(\boldsymbol{\theta})$ is our prior assumption about the distribution of the parameters (often a neutral

assessment where all parameters are equally likely). $\mathbb{P}$(Data) represents the overall probability of observing the data. Often, this is just a normalizing constant and can be inferred from the form of the posterior distribution. In other circumstances, this quantity must be dealt with in some concrete way.

7.2 The Law of Total Probability

Another use of conditional probability is to separate the computation of the probability of an event into a number of conditional problems that are often easier to compute. Suppose that $\{F_i\}_{i=1}^n$ is a collection of events that completely partition the sample space. That is $F_i \cap F_j = \emptyset$ for all indices $i \neq j$ and the union of all F_i yields the entire sample space. For any event E, the set of events $\{E \cap F_i\}_{i=1}^n$ are necessarily pairwise disjoint themselves while their union gives all of event E.

$$\begin{aligned}
\mathbb{P}(E) &= \mathbb{P}\left(\bigcup_i E \cap F_i\right) \\
&= \sum_{i=1}^n \mathbb{P}(E \cap F_i) \\
&= \sum_{i=1}^n \frac{\mathbb{P}(E \cap F_i)}{\mathbb{P}(F_i)} \mathbb{P}(F_i) \\
&= \sum_{i=1}^n \mathbb{P}(E|F_i)\mathbb{P}(F_i)
\end{aligned}$$

This result is known as the *law of total probability*. Note that it can be extended to a countably infinite sequence of partitioning events F_i.

7.3 A Sample Problem

Consider a game where we randomly select one of two coins labeled A and B (we will write $C = A$ for the event that coin A was selected and likewise for B). Note that we do not know which coin was selected, but we assume that the coins have an equal probability of being selected. We are also unsure if either coin is fair or is biased in some way. Once selected, we flip the coin 10 times and observe the number of heads in the outcome. Suppose we play this game 5 times and see the following outcomes. The model parameters we

Round	Outcome	Number of Heads
1	$HTTTHHTHTH$	5
2	$HHHHTHHHHH$	9
3	$HTHHHHHTHH$	8
4	$HTHTTTHHTT$	4
5	$THHHTHHHTH$	7

Figure 19: Coin Flip Outcomes

are unsure about are $\boldsymbol{\theta} = \{\theta_A, \theta_B\}$ which are the probabilities that the coin will turn up heads on any given flip.

$$\mathbb{P}(H|C = A) = \theta_A$$
$$\mathbb{P}(H|C = B) = \theta_B$$

Given the table above, we might suspect that at least one of the coins is biased towards heads. Suppose we initially take $\theta_A = 0.6$ and $\theta_B = 0.5$. How should we update these probabilities to reflect the data we have? First, consider the likelihood that each round was played with either coin. For Round 1, we have the following computation.

$$\mathbb{P}(C = A \mid \text{Data}_1) = \frac{\mathbb{P}(\text{ Data}_1|C = A)\mathbb{P}(C = A)}{\mathbb{P}(\text{ Data}_1)}$$

Note that $\mathbb{P}(\text{ Data}_1|C = A)$ is given by the binomial distribution using our prior assumption about θ_A.

$$\mathbb{P}(\text{ Data}_1|C = A) = \binom{10}{5}\theta_A^5(1-\theta_A)^5$$

For the overall probability of the data, we use the law of total probability.

$$\begin{aligned}\mathbb{P}(\text{ Data}_1) &= \mathbb{P}(\text{ Data}_1|C = A)\mathbb{P}(C = A) + \mathbb{P}(\text{ Data}_1|C = B)\mathbb{P}(C = B) \\ &= \binom{10}{5}\theta_A^5(1-\theta_A)^5\mathbb{P}(C = A) + \binom{10}{5}\theta_B^5(1-\theta_B)^5\mathbb{P}(C = B)\end{aligned}$$

If we assume that the coins were equally likely to be selected (that is, we assume $\mathbb{P}(C = A) = \mathbb{P}(C = B)$), then we have

$$\begin{aligned}\mathbb{P}(C = A|\text{ Data}_1) &= \frac{\mathbb{P}(\text{ Data}_1|C = A)\mathbb{P}(C = A)}{\mathbb{P}(\text{ Data}_1)} \\ &= \frac{\theta_A^5(1-\theta_A)^5}{\theta_A^5(1-\theta_A)^5 + \theta_B^5(1-\theta_B)^5} \approx 0.449\end{aligned}$$

using our prior assumption about the parameters. Likewise,

$$\begin{aligned}\mathbb{P}(C=B|\text{ Data}_1) &= \frac{\mathbb{P}(\text{ Data}_1|C=B)\mathbb{P}(C=B)}{\mathbb{P}(\text{ Data}_1)} \\ &= \frac{\theta_B^5(1-\theta_B)^5}{\theta_A^5(1-\theta_A)^5+\theta_B^5(1-\theta_B)^5} \approx 0.551.\end{aligned}$$

Note that we must have

$$\mathbb{P}(C=A|\text{ Data}_1)+\mathbb{P}(C=B|\text{ Data}_1)=1$$

since we must have selected one of the two coins. If n is the total number of flips in a round and k the number of heads, we see that for any particular round

$$\begin{aligned}\mathbb{P}(C=A|\text{ Data }) &= \frac{\theta_A^k(1-\theta_A)^{n-k}}{\theta_A^k(1-\theta_A)^{n-k}+\theta_B^k(1-\theta_B)^{n-k}}, \\ \mathbb{P}(C=B|\text{ Data }) &= \frac{\theta_B^k(1-\theta_B)^{n-k}}{\theta_A^k(1-\theta_A)^{n-k}+\theta_B^k(1-\theta_B)^{n-k}}.\end{aligned}$$

Computing these values for our 5 rounds gives the following table. Given

Round	Number of Heads	$\mathbb{P}(C=A\|\text{ Data}_i)$	$\mathbb{P}(C=B\|\text{ Data}_i)$
1	5	0.449	0.551
2	9	0.805	0.195
3	8	0.733	0.267
4	4	0.352	0.648
5	7	0.647	0.353

Figure 20: Coin Probabilities from Prior Assumptions on $\boldsymbol{\theta}$

the probabilities above, we can attribute some portion of the heads in each round to either coin. As an example, we expect that 80.5% of the 9 heads in round 2 came from coin A. That is, 7.245 of the heads in round 2 are attributable to coin A while 1.755 are from coin B. Carrying this out for all 5 rounds gives the following table.

We can now use this information to update our assessment of the model parameters $\boldsymbol{\theta}=\{\theta_A,\theta_B\}$. The idea is that we look at the average number of

Round	$\mathbb{P}(C = A \mid \text{Data}_i)$	$\mathbb{P}(C = B \mid \text{Data}_i)$	Heads from A	Heads from B
1	0.449	0.551	2.245	2.755
2	0.805	0.195	7.245	1.755
3	0.733	0.267	5.864	2.136
4	0.352	0.648	1.408	2.592
5	0.647	0.353	4.529	2.471

Figure 21: Heads Attribution from Prior Assumptions on $\boldsymbol{\theta}$

attributed heads in the 10 coin flips across the 5 rounds. However, we only have a *probability* that each coin was used in a particular round. So instead of dividing by 5, we divide by the sum of the probabilities that each coin was used. This should give a more accurate idea of how many times the coin was actually selected.

$$\theta_A = \frac{1}{10}\left(\frac{2.245 + 7.245 + 5.864 + 1.408 + 4.529}{0.449 + 0.805 + 0.733 + 0.352 + 0.647}\right) \approx 0.713$$
$$\theta_B = \frac{1}{10}\left(\frac{2.755 + 1.755 + 2.136 + 2.592 + 2.471}{0.551 + 0.195 + 0.267 + 0.648 + 0.353}\right) \approx 0.581$$

We can now repeat the procedure with our updated values of $\boldsymbol{\theta}$ until they converge (or at least until the amount of change falls below some threshold). On the second pass, we find the following.

Round	$\mathbb{P}(C = A \mid \text{Data}_i)$	$\mathbb{P}(C = B \mid \text{Data}_i)$	Heads from A	Heads from B
1	0.296	0.704	1.478	3.522
2	0.812	0.188	7.310	1.690
3	0.707	0.293	5.656	2.344
4	0.190	0.810	0.759	3.241
5	0.574	0.426	4.018	2.982

Figure 22: Heads Attribution for Second Pass

$$\theta_A = \frac{1}{10}\left(\frac{1.478 + 7.310 + 5.656 + 0.759 + 4.018}{0.296 + 0.812 + 0.707 + 0.190 + 0.574}\right) \approx 0.745$$
$$\theta_B = \frac{1}{10}\left(\frac{3.522 + 1.690 + 2.344 + 3.241 + 2.982}{0.704 + 0.188 + 0.293 + 0.810 + 0.426}\right) \approx 0.569$$

If we repeat this process enough times, we see that the probabilities stabilize to roughly

$$\theta_A = \mathbb{P}(H|C = A) \approx 0.797,$$
$$\theta_B = \mathbb{P}(H|C = B) \approx 0.520.$$

Once we have these values, we can make a judgement about which coin was used for each round by selecting the coin that has the highest probability. For this example, we decide that coin A was most likely used on rounds 2, 3, and 5 while coin B was used for rounds 1 and 4.

7.4 Maximum Likelihood (MLE) and Maximum a Posteriori (MAP) Estimators

It turns out that our considerations from the coin toss problem can be generalized to a large variety of situations. Ultimately, this will lead us the Expectation Maximization Algorithm. Before we dive into this subject, we should define a few concepts that are crucial for the theory.

Suppose that our model has parameters $\boldsymbol{\Theta} = \{\theta_1, \theta_2, \cdots, \theta_n\}$ and that we have a probability density function that tells us the likelihood of an observation $\mathbf{X}$ *given the parameters* $\boldsymbol{\theta}$. We denote this distribution function by $p_{\boldsymbol{X}|\boldsymbol{\Theta}}(\boldsymbol{x}|\boldsymbol{\theta})$ where $\boldsymbol{x}$ is a generic observation and $\boldsymbol{\theta}$ a generic set of parameters.

Given a particular observation $\boldsymbol{X}$, what values should we assign to the parameters. A logical choice is the value of the generic parameters $\boldsymbol{\theta}$ that makes the data most likely. That is, we choose an estimate $\hat{\boldsymbol{\Theta}}$ that maximizes the value of the probability density function *given the value of the observation* $\boldsymbol{X}$.

$$\hat{\boldsymbol{\Theta}} = \arg\max_{\boldsymbol{\theta}} p_{\boldsymbol{X}|\boldsymbol{\Theta}}(\boldsymbol{X}|\boldsymbol{\theta})$$

This estimate for the true parameters $\boldsymbol{\Theta}$ is known as the *maximum likelihood estimator*.

Since we often have to resort to numeric optimization to find these estimators, it is more convenient to work with the logarithm of the probability distribution function (as the values of the distribution function might be fairly small). If we define

$$\ell(\boldsymbol{\theta}; \boldsymbol{x}) = \ln p_{\boldsymbol{X}|\boldsymbol{\Theta}}(\boldsymbol{x}|\boldsymbol{\theta}),$$

then we also have

$$\hat{\boldsymbol{\Theta}} = \arg\max_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}; \boldsymbol{X})$$

for any observation $\boldsymbol{X}$ since the natural logarithm is monotonically increasing.

If we take a Bayesian approach and assume that there is a prior probability distribution $p(\boldsymbol{\theta})$ on the parameters, then

$$p(\boldsymbol{\theta}|\boldsymbol{X}) = \frac{p_{\boldsymbol{X}|\boldsymbol{\Theta}}(\boldsymbol{X}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\boldsymbol{X})}.$$

We can then find the set of parameters $\boldsymbol{\theta}$ that maximize this quantity. Note that the overall probability of the observation, $p(\boldsymbol{X})$, is independent of the value of the parameters. Hence, we can take

$$\hat{\boldsymbol{\Theta}} = \arg\max_{\boldsymbol{\theta}} p_{\boldsymbol{X}|\boldsymbol{\Theta}}(\boldsymbol{X}|\boldsymbol{\theta})p(\boldsymbol{\theta}).$$

This estimate for the true parameters $\boldsymbol{\Theta}$ is known as the *maximum a posteriori estimator*. Note the we can recover the maximum likelihood estimator if we take a uniform (i.e. constant) prior for $p(\boldsymbol{\theta})$. Note that if we take the natural logarithm we find

$$\ell(\boldsymbol{\theta}; \boldsymbol{x}) = \ln p_{\boldsymbol{X}|\boldsymbol{\Theta}}(\boldsymbol{x}|\boldsymbol{\theta}) + \ln p(\boldsymbol{\theta})$$

and

$$\hat{\boldsymbol{\Theta}} = \arg\max_{\boldsymbol{\theta}} \left(\ln p_{\boldsymbol{X}|\boldsymbol{\Theta}}(\boldsymbol{X}|\boldsymbol{\theta}) + \ln p(\boldsymbol{\theta}) \right).$$

7.5 The EM Algorithm

The generalization of the procedure we used in the coin flip example is known as the Expectation Maximization (EM) Algorithm. One of the principal advantages of this algorithm is that it can allow us to estimate model parameters even if there latent variables in the data. To understand the setup, let $\boldsymbol{Z}$ represent a complete observation of the data which we break down into

$$\boldsymbol{Z} = (\boldsymbol{X}, \boldsymbol{Y}),$$

where $\boldsymbol{X}$ is the portion of the complete observation we actually have data for while $\boldsymbol{Y}$ represents the latent variables we have not observed. In order to compute maximum likelihoods, we would first have to sum over all

possibilities for the latent variables:

$$p_{\boldsymbol{X}|\boldsymbol{\Theta}}(\boldsymbol{x}|\boldsymbol{\theta}) = \int p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(\boldsymbol{x}, \boldsymbol{y}|\boldsymbol{\theta}) p_{\boldsymbol{Y}}(\boldsymbol{y}) d\boldsymbol{y}$$

where $p_{\boldsymbol{Y}}(\boldsymbol{y})$ represents some (prior) distribution on the latent variables. We would then need to maximize this quantity (or its logarithm) with respect to the generic parameters $\boldsymbol{\theta}$. The issue is that we may not have any certainty about the distribution of the latent variables. Even if we do, the marginal distribution might be difficult to compute, and the resulting function could be difficult to maximize.

Instead, we perform an iterative procedure. We make an initial guess for the parameters $\boldsymbol{\theta}_0$. We then compute the expected value of $p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(\boldsymbol{x}, \boldsymbol{y}|\boldsymbol{\theta})$ given the observed data $\boldsymbol{X}$ and using $\boldsymbol{\theta}_0$ to compute the expectation over the unobserved values $\boldsymbol{Y}$.

$$\begin{aligned} Q(\boldsymbol{\theta}|\boldsymbol{\theta}_0) &= \mathbb{E}_{\boldsymbol{\theta}_0}\left[p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(\boldsymbol{x}, \boldsymbol{y}|\boldsymbol{\theta})|\boldsymbol{x} = \boldsymbol{X}\right] \\ &= \int p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(\boldsymbol{X}, \boldsymbol{y}|\boldsymbol{\theta}) p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(\boldsymbol{y}|\boldsymbol{x} = \boldsymbol{X}, \boldsymbol{\theta}_0) d\boldsymbol{y} \end{aligned}$$

Once we have computed this quantity, we choose

$$\boldsymbol{\theta}_1 = \arg\max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}|\boldsymbol{\theta}_0).$$

We repeat this process until the parameters converge.

To summarize, the EM Algorithm consists of two basic steps.

0) **Initialization:** Randomly select the initial parameters $\boldsymbol{\theta}_0$.

1) **The Expectation Step:** Compute

$$Q(\boldsymbol{\theta}|\boldsymbol{\theta}_i) = \mathbb{E}_{\boldsymbol{\theta}_i}\left[p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(\boldsymbol{x}, \boldsymbol{y}|\boldsymbol{\theta})|\boldsymbol{x} = \boldsymbol{X}\right]$$

   using $\boldsymbol{\theta}_i$ to take the expectation over the latent variables $\boldsymbol{y}$.

2) **The Maximization Step:** Update the parameters via

$$\boldsymbol{\theta}_{i+1} = \arg\max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}|\boldsymbol{\theta}_i).$$

As mentioned in the previous section, the algorithm is usually implemented with

$$Q(\boldsymbol{\theta}|\boldsymbol{\theta}_i) = \mathbb{E}_{\boldsymbol{\theta}_i}\left[\ln p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(\boldsymbol{x}, \boldsymbol{y}|\boldsymbol{\theta})|\boldsymbol{x} = \boldsymbol{X}\right]$$

since this handles independent events in a particularly simple fashion.

In order to make the notation clearer, we will re–examine the coin flip example in these terms. First, a full observation for any round of the game would be the coin used ($C = A$ or $C = B$) along with the number of heads observed in the round: $\boldsymbol{Z} = (H, C)$ where H is the number of heads in the 10 tosses and C is the coin selected. Since we are not told which coin was selected, C is the latent variable in the experiment (we assume each coin has an equal chance of being selected, however). The model parameters are $\boldsymbol{\Theta} = (\theta_A, \theta_B)$ which give the probabilities of a coin toss coming up heads for each coin. Recall that we set our initial guess for these parameters as $\boldsymbol{\theta}_0 = (0.6, 0.5)$. The (discrete) probability distribution function is

$$\begin{aligned}
p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(h, c|\boldsymbol{\theta}) &= \frac{1}{2}\left(\binom{10}{h}\theta_A^h(1-\theta_A)^{10-h}\right)\mathbb{1}_A(c) + \frac{1}{2}\left(\binom{10}{h}\theta_B^h(1-\theta_B)^{10-h}\right)\mathbb{1}_B(c) \\
&= \frac{1}{2}\binom{10}{h}\left(\theta_A^h(1-\theta_A)^{10-h}\mathbb{1}_A(c) + \theta_B^h(1-\theta_B)^{10-h}\mathbb{1}_B(c)\right)
\end{aligned}$$

where $\mathbb{1}_A(c)$ is 1 if $c = A$ and 0 otherwise (and the analogous function for $\mathbb{1}_B$). The conditional probability of the coin given the number of flips is

$$\begin{aligned}
p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(c|h = H, \boldsymbol{\theta}) &= \frac{\frac{1}{2}\binom{10}{H}\left(\theta_A^H(1-\theta_A)^{10-H}\mathbb{1}_A(c) + \theta_B^H(1-\theta_B)^{10-H}\mathbb{1}_B(c)\right)}{\mathbb{P}(h = H)} \\
&= \frac{\frac{1}{2}\binom{10}{H}\left(\theta_A^H(1-\theta_A)^{10-H}\mathbb{1}_A(c) + \theta_B^H(1-\theta_B)^{10-H}\mathbb{1}_B(c)\right)}{\frac{1}{2}\binom{10}{H}\left(\theta_A^H(1-\theta_A)^{10-H} + \theta_B^H(1-\theta_B)^{10-H}\right)} \\
&= \frac{\theta_A^H(1-\theta_A)^{10-H}\mathbb{1}_A(c) + \theta_B^H(1-\theta_B)^{10-H}\mathbb{1}_B(c)}{\theta_A^H(1-\theta_A)^{10-H} + \theta_B^H(1-\theta_B)^{10-H}} \\
&= \left(\frac{\theta_A^H(1-\theta_A)^{10-H}}{\theta_A^H(1-\theta_A)^{10-H} + \theta_B^H(1-\theta_B)^{10-H}}\right)\mathbb{1}_A(c) \\
&\qquad + \left(\frac{\theta_B^H(1-\theta_B)^{10-H}}{\theta_A^H(1-\theta_A)^{10-H} + \theta_B^H(1-\theta_B)^{10-H}}\right)\mathbb{1}_B(c)
\end{aligned}$$

This should look very familiar if we break it down by choice of coin.

$$p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(c = A|h = H, \boldsymbol{\theta}) = \frac{\theta_A^H(1-\theta_A)^{10-H}}{\theta_A^H(1-\theta_A)^{10-H} + \theta_B^H(1-\theta_B)^{10-H}}$$
$$p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(c = B|h = H, \boldsymbol{\theta}) = \frac{\theta_B^H(1-\theta_B)^{10-H}}{\theta_A^H(1-\theta_A)^{10-H} + \theta_B^H(1-\theta_B)^{10-H}}$$

If we have a *single* observation for this experiment, then

$$\begin{aligned}
Q(\boldsymbol{\theta}|\boldsymbol{\theta}_0) &= \mathbb{E}_{\boldsymbol{\theta}_0}\left[p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(h, c|\boldsymbol{\theta})|h = H\right] \\
&= p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(H, A|\boldsymbol{\theta})p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(A|h = H, \boldsymbol{\theta}_0) + p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(H, B|\boldsymbol{\theta})p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(B|h = H, \boldsymbol{\theta}_0) \\
&= \frac{1}{2}\left(\binom{10}{H}\theta_A^H(1-\theta_A)^{10-H}\right)\frac{(0.6)^H(0.4)^{10-H}}{(0.6)^H(0.4)^{10-H} + (0.5)^H(0.5)^{10-H}} \\
&\qquad + \frac{1}{2}\left(\binom{10}{H}\theta_B^H(1-\theta_B)^{10-H}\right)\frac{(0.5)^H(0.5)^{10-H}}{(0.6)^H(0.4)^{10-H} + (0.5)^H(0.5)^{10-H}}
\end{aligned}$$

To maximize this, we need the partial derivatives with respect to the parameters. If $0 < H < 10$, then

$$\begin{aligned}
\frac{\partial Q}{\partial \theta_A} &= \mathcal{C}_A\theta_A^{H-1}(1-\theta_A)^{9-H}\left(H - 10\theta_A\right) = 0, \\
\frac{\partial Q}{\partial \theta_B} &= \mathcal{C}_B\theta_B^{H-1}(1-\theta_B)^{9-H}\left(H - 10\theta_B\right) = 0,
\end{aligned}$$

where $\mathcal{C}_A$ and $\mathcal{C}_B$ are coefficients independent of $\boldsymbol{\theta}$. This would give

$$\boldsymbol{\theta}_1 = \arg\max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}|\boldsymbol{\theta}_0) = \left(\frac{H}{10}, \frac{H}{10}\right)$$

which is clearly consistent with a single observation (note that these values also work in the extreme cases $H = 0, 10$). We do not need to perform any more rounds since these values will be the same regardless of the coefficients.

For multiple observations, we assume independence and must compute the expected value of

$$\prod_{i=1}^{n} p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(h_i, c_i|\boldsymbol{\theta}).$$

It is certainly to our advantage to use log–likelihood here since

$$\begin{aligned}\ln\left(\prod_{i=1}^{n} p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(h_i, c_i|\boldsymbol{\theta})\right) &= \sum_{i=1}^{n} \ln p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(h_i, c_i|\boldsymbol{\theta}) \\ &= \sum_{i=1}^{n}\Bigg[\ln\left(\frac{1}{2}\binom{10}{h_i}\right) + \Big(h_i \ln(\theta_A) + (10-h_i)\ln(1-\theta_A)\Big)\mathbb{1}_A(c_i) \\ &\qquad + \Big(h_i \ln(\theta_B) + (10-h_i)\ln(1-\theta_B)\Big)\mathbb{1}_B(c_i)\Bigg].\end{aligned}$$

This means that

$$Q(\boldsymbol{\theta}|\boldsymbol{\theta}_0) = \sum_{i=1}^{n} \mathbb{E}_{\boldsymbol{\theta_0}}\left[\ln p_{\boldsymbol{Z}|\boldsymbol{\Theta}}(h_i, c_i|\boldsymbol{\theta})\middle| h_i = H_i\right]$$

(again, using independence of the rounds). Since each term in the sum is of the form

$$\begin{aligned}&\mathcal{C}_{A,i}\left[\ln\left(\frac{1}{2}\binom{10}{H_i}\right) + \Big(H_i \ln(\theta_A) + (10-H_i)\ln(1-\theta_A)\Big)\right]\mathbb{1}_A(c_i) \\ &\qquad + \mathcal{C}_{B,i}\left[\ln\left(\frac{1}{2}\binom{10}{H_i}\right) + \Big(H_i \ln(\theta_B) + (10-H_i)\ln(1-\theta_B)\Big)\right]\mathbb{1}_B(c_i)\end{aligned}$$

we have partial derivatives of the form

$$\begin{aligned}\frac{\partial Q}{\partial \theta_A} &= \sum_{i=1}^{n} \mathcal{C}_{A,i}\frac{H_i - 10\theta_A}{\theta_A(1-\theta_A)}, \\ \frac{\partial Q}{\partial \theta_B} &= \sum_{i=1}^{n} \mathcal{C}_{B,i}\frac{H_i - 10\theta_B}{\theta_B(1-\theta_B)}.\end{aligned}$$

Setting these derivatives equal to zero gives $\boldsymbol{\theta}_1$ with components below.

$$\begin{aligned}\theta_A &= \frac{\sum_{i=1}^{n}\mathcal{C}_{A,i}H_i}{10\sum_{i=1}^{n}\mathcal{C}_{A,i}} \\ \theta_B &= \frac{\sum_{i=1}^{n}\mathcal{C}_{B,i}H_i}{10\sum_{i=1}^{n}\mathcal{C}_{B,i}}\end{aligned}$$

Tracing back through the definitions, we see that

$$\mathcal{C}_{A,i} = \frac{(0.6)^{H_i}(0.4)^{10-H_i}}{(0.6)^{H_i}(0.4)^{10-H_i} + (0.5)^{H_i}(0.5)^{10-H_i}},$$
$$\mathcal{C}_{B,i} = \frac{(0.5)^{H_i}(0.5)^{10-H_i}}{(0.6)^{H_i}(0.4)^{10-H_i} + (0.5)^{H_i}(0.5)^{10-H_i}}.$$

But this is precisely the procedure we employed to update the probabilities when we originally thought about this experiment!

8 Lecture 10: Gaussian Mixture Models

April 22, 2024

8.1 Introduction to Gaussian Mixture Models (GMM)

In k–Means, we assume that there are some number of *spherical* clusters. We find the centroids by the obvious iterative procedure, and we can repeat this process for many choices of the parameter k (i.e. the number of clusters) in an attempt to find a decent fit.

In many datasets, there may be non-spherical clusters that we wish to identify. In the dataset displayed below, there appear to be three clusters. The clusters in the lower left and upper center seem reasonably spherical,

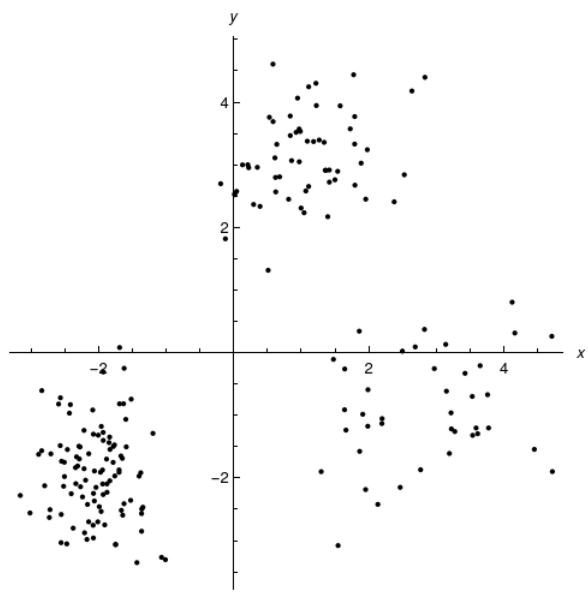


Figure 23: Dataset with 3 Clusters

but the other seems to have a more elliptic shape. While k–Means is likely to find the correct centroids, it will not directly give us any indication about the shapes of the clusters.

Gaussian Mixture Models are a generalization of the k–Means algorithm that attempt to fit linear combinations of normal random variables to the data. In particular, it attempts to find three sets of parameters to describe

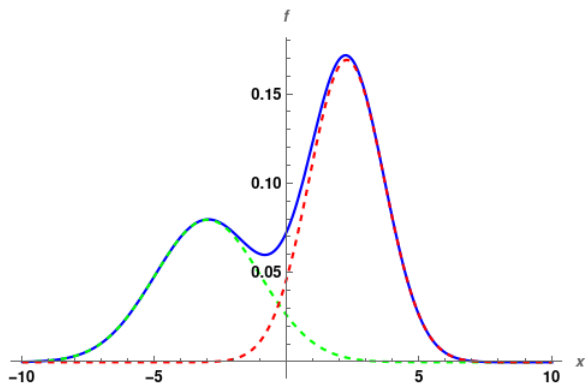


Figure 24: Mixture of Two Gaussians in One–Dimension

the data.

1) $\boldsymbol{\pi} = [\pi_1, \pi_2, \cdots \pi_K]$

   These values represent the probability that a randomly drawn sample will belong to the corresponding cluster. Since these are probabilities, they must be non-negative and

$$\pi_1 + \pi_2 + \cdots + \pi_K = 1.$$

   Below, any π appearing without a subscript represents the traditional value (i.e. $\pi \approx 3.14159$)!

2) $\vec{\boldsymbol{\mu}} = [\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \cdots \boldsymbol{\mu}_K]$

   Each $\boldsymbol{\mu}_i$ is the centroid of a cluster. Note that each of these is a vector of dimension m (where m is the number of features).

3) $\vec{\boldsymbol{\Sigma}} = [\boldsymbol{\Sigma}_1, \boldsymbol{\Sigma}_2, \cdots \boldsymbol{\Sigma}_K]$

   Each $\boldsymbol{\Sigma}_i$ is the $m \times m$ covariance matrix associated to the Gaussian distribution for the associated cluster. Since this is a covariance matrix, it will be symmetric and positive definite (in particular, its determinant $|\boldsymbol{\Sigma}_i|$ will be positive).

The multivariate normal distribution gives the probability distribution

for a data point *given that it is a member of that cluster.*

$$\begin{aligned} p(\boldsymbol{x}|C=k) &= \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \\ &= \frac{1}{\sqrt{(2\pi)^m|\boldsymbol{\Sigma}_k|}} \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_k)^T\boldsymbol{\Sigma}_k^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_k)\right) \end{aligned}$$

Note that we have used the random variable C to denote the cluster label. Since we have a mixture of Gaussians, the overall probability distribution is given by

$$p(\boldsymbol{x}) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

The question we need to address is how to discover the parameters of the mixture model from the data $\boldsymbol{X} = \{\boldsymbol{x}_n\}_{n=1}^N$. Assuming independence, we can write the overall probability distribution for the dataset given the parameters as

$$\begin{aligned} p(\boldsymbol{X}|\boldsymbol{\pi}, \vec{\boldsymbol{\mu}}, \vec{\boldsymbol{\sigma}}) &= \prod_{n=1}^{N} p(\boldsymbol{x}_n|\boldsymbol{\pi}, \vec{\boldsymbol{\mu}}, \vec{\boldsymbol{\sigma}}) \\ &= \prod_{n=1}^{N} \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \end{aligned}$$

As usual, we try to identify the parameters that maximize the log–likelihood function.

$$\begin{aligned} \ln p(\boldsymbol{X}|\boldsymbol{\pi}, \vec{\boldsymbol{\mu}}, \vec{\boldsymbol{\sigma}}) &= \ln\left(\prod_{n=1}^{N} \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)\right) \\ &= \sum_{n=1}^{N} \ln\left(\sum_{k=1}^{K} \pi_k \, \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)\right) \end{aligned}$$

Unfortunately, this is far too complex to optimize directly. So, we will implement the Expectation–Maximization Algorithm to try and identify the best choices.

First, note that Bayesian analysis gives us the following.

$$\begin{aligned} \gamma_{i,j} = p(C=j|\boldsymbol{x_i}) &= \frac{p(\boldsymbol{x_i}|C=j)p(C=j)}{p(\boldsymbol{x_i})} \\ &= \frac{\pi_j \, \mathcal{N}(\boldsymbol{x_i}|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}{\sum_{k=1}^{K} \pi_k \, \mathcal{N}(\boldsymbol{x_i}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)} \end{aligned}$$

We think of $\gamma_{i,j}$ as representing the probability that data point $\boldsymbol{x_i}$ belongs to cluster i given the parameters $\boldsymbol{\pi}, \vec{\boldsymbol{\mu}}$, and $\vec{\boldsymbol{\Sigma}}$. These are usually referred to as *responsibility* coefficients. These coefficients play a key role in the partial derivatives of the log–likelihood function.

$$\begin{aligned}
\nabla_{\boldsymbol{\mu}_\ell} \ln p(\boldsymbol{X}|\boldsymbol{\pi}, \vec{\boldsymbol{\mu}}, \vec{\boldsymbol{\sigma}}) &= \sum_{n=1}^{N} \frac{\pi_\ell \, \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_\ell, \boldsymbol{\Sigma}_\ell)}{\sum_{k=1}^{K} \pi_k \, \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)} \boldsymbol{\Sigma}_\ell^{-1}(\boldsymbol{x}_n - \boldsymbol{\mu}_\ell) \\
&= \boldsymbol{\Sigma}_\ell^{-1} \sum_{n=1}^{N} \gamma_{n,\ell}(\boldsymbol{x}_n - \boldsymbol{\mu}_\ell)
\end{aligned}$$

Since we need these derivatives to be zero, we must have

$$\begin{aligned}
0 &= \sum_{n=1}^{N} \gamma_{n,\ell}(\boldsymbol{x}_n - \boldsymbol{\mu}_\ell) \\
&= \sum_{n=1}^{N} \gamma_{n,\ell}\boldsymbol{x}_n - \left(\sum_{n=1}^{N} \gamma_{n,\ell}\right)\boldsymbol{\mu}_\ell \\
\boldsymbol{\mu}_\ell &= \frac{1}{N_\ell} \sum_{n=1}^{N} \gamma_{n,\ell}\boldsymbol{x}_n
\end{aligned}$$

where $N_\ell = \sum_{n=1}^{N} \gamma_{n,\ell}$. We can think of N_ℓ as the total number of data points belonging to cluster ℓ. The derivative with respect to the covariance matrix is trickier, but it yields the following.

$$\begin{aligned}
0 &= \nabla_{\boldsymbol{\Sigma}_\ell} \ln p(\boldsymbol{X}|\boldsymbol{\pi}, \vec{\boldsymbol{\mu}}, \vec{\boldsymbol{\sigma}}) \\
&= -\frac{1}{2} \sum_{n=1}^{N} \gamma_{n,\ell} \left(\boldsymbol{\Sigma}_\ell^{-1} - \boldsymbol{\Sigma}_\ell^{-1}(\boldsymbol{x}_n - \boldsymbol{\mu}_\ell)(\boldsymbol{x}_n - \boldsymbol{\mu}_\ell)^T \boldsymbol{\Sigma}_\ell^{-1}\right) \\
N_\ell \boldsymbol{\Sigma}_\ell^{-1} &= \boldsymbol{\Sigma}_\ell^{-1} \left(\sum_{n=1}^{N} \gamma_{n,\ell}(\boldsymbol{x}_n - \boldsymbol{\mu}_\ell)(\boldsymbol{x}_n - \boldsymbol{\mu}_\ell)^T\right) \boldsymbol{\Sigma}_\ell^{-1} \\
\boldsymbol{\Sigma}_\ell &= \frac{1}{N_\ell} \sum_{n=1}^{N} \gamma_{n,\ell}(\boldsymbol{x}_n - \boldsymbol{\mu}_\ell)(\boldsymbol{x}_n - \boldsymbol{\mu}_\ell)^T
\end{aligned}$$

To begin, we randomly initialize $\vec{\boldsymbol{\mu}}$ and $\vec{\boldsymbol{\sigma}}$ and set $\pi_j = \frac{1}{K}$ (all clusters are equally likely, initially). The Expectation Step uses the randomized means

and covariance matrices to update the responsibility coefficients.

$$\gamma_{i,j} \longleftarrow \frac{\pi_j \, \mathcal{N}(\boldsymbol{x_i}|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}{\sum_{k=1}^{K} \pi_k \, \mathcal{N}(\boldsymbol{x_i}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}$$

The Maximization Step updates the probabilities, means, and covariance matrices.

$$\begin{aligned}
\pi_j &\longleftarrow \frac{N_j}{N} \\
\boldsymbol{\mu}_j &\longleftarrow \frac{1}{N_j} \sum_{n=1}^{N} \gamma_{n,j} \boldsymbol{x}_n \\
\boldsymbol{\Sigma}_j &\longleftarrow \frac{1}{N_j} \sum_{n=1}^{N} \gamma_{n,j} (\boldsymbol{x}_n - \boldsymbol{\mu}_j)(\boldsymbol{x}_n - \boldsymbol{\mu}_j)^T
\end{aligned}$$

We repeat these step until convergence (or until the change in the log–likelihood function falls below some threshold).

Once we have computed probabilities $\boldsymbol{\pi}$, means $\vec{\boldsymbol{\mu}}$, and covariance matrices $\vec{\boldsymbol{\Sigma}}$, we can classify new data points by considering likelihoods. For data point $\boldsymbol{x}$, we compute the values

$$L_j = \pi_j \, \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$$

for all $1 \leq j \leq K$. Often, the logarithms of these values are used instead in case the likelihoods are very small. We classify the point as belonging to the cluster k for which L_k is the largest. Running this procedure with three clusters on the data from the beginning of the section yields the following classification.

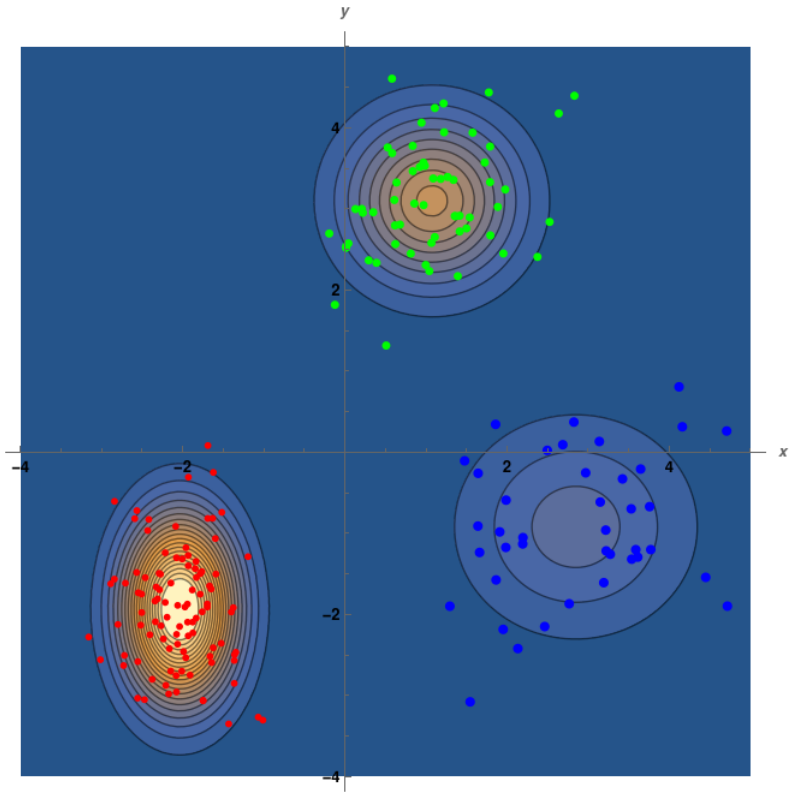


Figure 25: Gaussian Mixture Model with 3 Clusters

References

[1] Bishop C. and Bishop H., *Deep Learning: Foundations and Concepts*, Switzerland: Springer Nature, 2024.

[2] Goodfellow, I., Bengio, Y. and Courville, A., *Deep Learning*, Cambridge, MA: The MIT Press, 2016.